Sonar Pulse Detection with a CNN–LSTM Sequence Model\[4pt]

A Complete Walkthrough of Methods, Mathematics and Implementation

Contents

How to read this document

Every method in this project is presented in the same four parts:

1. **Why** — the problem the method solves, and why the obvious alternatives fall short.
2. **How it works** — the mathematics, built up step by step. No formula appears before the idea behind it.
3. **How it is implemented** — the actual code from the repository, with each line mapped back to the mathematics.
4. **What it produces** — measured output. Every number in this document was produced by running the code; none are assumed.

Numbers that depend on your trained model (final accuracy, false-alarm rate, and so on) are **not** quoted, because they depend on the configuration your hyperparameter search selected. Where such a number belongs, the document names the script and the command that produces it. Everything else — resolution limits, interpolation accuracy, the behaviour of the metrics, the refinement precision on synthetic pulses — is measured and reported here.

Small utility functions are collected in the appendix rather than given the full treatment.

Part 0 — The problem

0.1 What the system has to do

A hydrophone records the ocean. Somewhere in hours of recording, an active sonar transmits pulses. The system must find them and describe them.

"Describe" means four numbers and a label per pulse:

Quantity	Symbol	Meaning
Start time	t_{start}	when the pulse arrives
Stop time	t_{stop}	when it ends
Start frequency	f_1	the frequency it begins at
Stop frequency	f_2	the frequency it ends at
Type	—	CW, LFM or HFM

Two features of the task shape every design decision that follows.

The number of pulses is not known in advance. A five-second window may contain none, one, or several. A plain classifier cannot express that: it produces a fixed-size output. This is what forces a *sequence* model, which emits pulses one at a time until it decides to stop.

The pulses are synthetic, the noise is real. There is no large annotated corpus of real sonar pulses, so pulses are simulated and injected into genuine ocean-noise recordings. The noise — shipping, wave motion, biology — is therefore real and unmodelled, while the signals are known exactly. This is what makes supervised training possible at all, and it is also the project's principal limitation, discussed in Part 10.

0.2 The three pulse types

All three are frequency-modulated tones of finite duration. They differ only in how the instantaneous frequency $f(t)$ evolves.

CW (continuous wave) — a constant tone:

$$f(t) = f_0$$

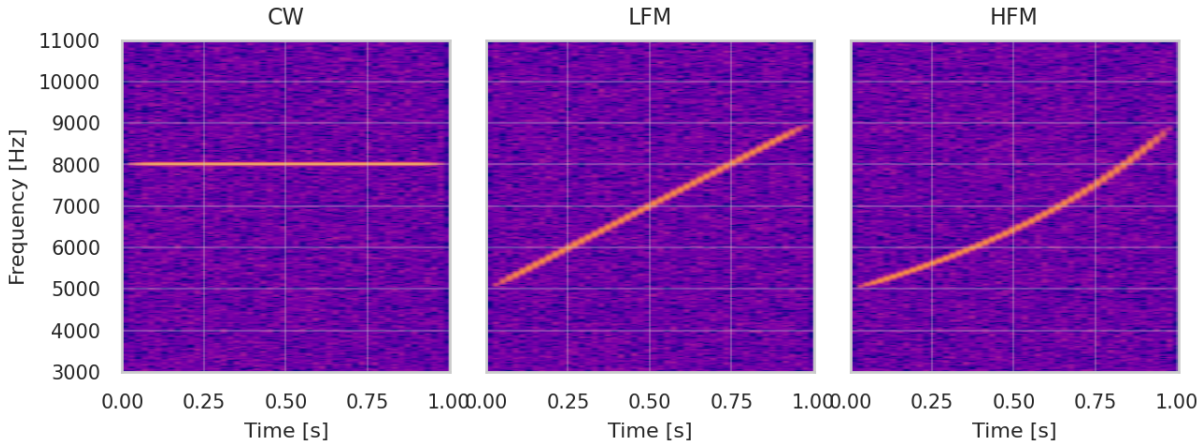

Figure 1: The three pulse types in the spectrogram. CW is a horizontal line; LFM and HFM both climb from 5 to 9 kHz.

LFM (linear frequency modulation) — frequency moves at a constant rate. Writing $\tau = (t - t_{\text{start}})/T$ for the fraction of the pulse elapsed, with T the duration:

$$f(\tau) = f_1 + (f_2 - f_1)\,\tau$$

HFM (hyperbolic frequency modulation) — the *period* moves at a constant rate, so the reciprocal of frequency is linear in time:

$$\frac{1}{f(\tau)} = \frac{1}{f_1} + \left(\frac{1}{f_2} - \frac{1}{f_1}\right)\tau \qquad \Longleftrightarrow \qquad f(\tau) = \left[\frac{1}{f_1} + \left(\frac{1}{f_2} - \frac{1}{f_1}\right)\tau\right]^{-1}$$

HFM is used in practice because it is tolerant to Doppler shift: a moving target compresses or stretches the signal in time, and a hyperbolic sweep is self-similar under that transformation in a way a linear one is not.

The hard discrimination is LFM against HFM. Both start at f_1 and end at f_2; they differ only in the path taken between those endpoints. How large is that difference? For a sweep from f_1 to f_2, the two curves are furthest apart near the middle of the pulse, where the gap is

$$\Delta_{\text{mid}} = \frac{(f_2 - f_1)^2}{2(f_1 + f_2)}$$

This is the difference between the arithmetic mean $\frac{1}{2}(f_1 + f_2)$, which the LFM passes through at mid-pulse, and the harmonic mean $2f_1f_2/(f_1 + f_2)$, which the HFM passes through.

Two consequences follow, and they matter for interpreting results later:

- For a 5000 to 9000 Hz sweep, $\Delta_{\text{mid}} = 571$ Hz (measured maximum separation: 584 Hz). Comfortably resolvable.
- For a **narrow** sweep, say 5000 to 5500 Hz, $\Delta_{\text{mid}} = 12$ Hz — smaller than one spectrogram bin. The two types are then *physically indistinguishable* in the data, no matter how good the model is.

The analysis script `snr_analysis.py` plots type accuracy against this curvature gap for exactly this reason: it separates "the model is weak" from "the information is not there".

0.3 Why a spectrogram, and why a sequence model

The raw waveform of a sonar pulse buried in noise looks like noise. What distinguishes a pulse is its *structure in frequency over time* — a line, or a curve, in the time–frequency plane. The

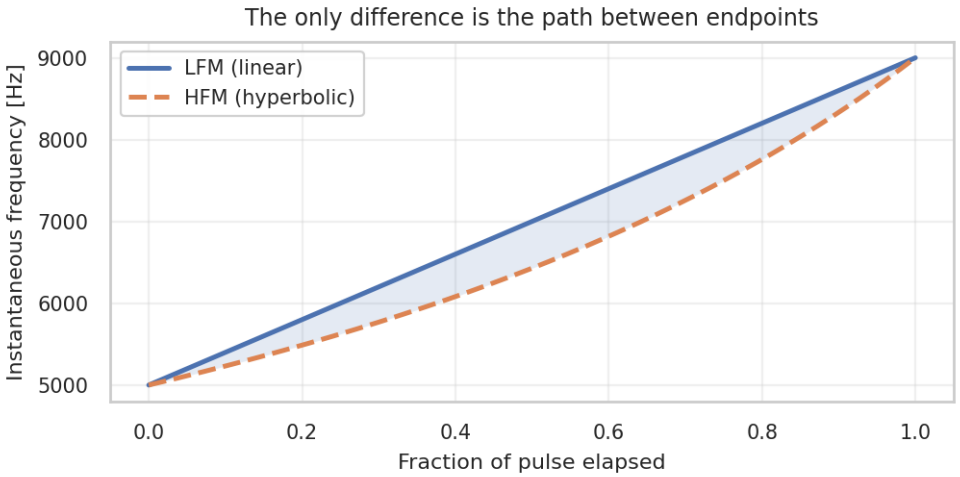

Figure 2: Instantaneous frequency of an LFM and an HFM sharing the same endpoints. The shaded area is all there is to tell them apart.

spectrogram makes that structure explicit, which is why it, and not the waveform, is the model's input.

Given a spectrogram, the task is close to object detection in an image: find each pulse and report its extent. Two standard approaches exist:

- **Anchor-based detection** (as in image detectors) — predefine candidate boxes and classify each. Effective, but it requires designing anchors and a non-maximum-suppression stage.
- **Sequence generation** — read the image, then emit objects one at a time until a stop token. This handles a variable count naturally and requires no anchors.

This project uses the second. The model is an *encoder–decoder*: the encoder compresses the spectrogram into a summary, and the decoder unrolls that summary into a list of pulses, ending with an explicit end-of-sequence (EOS) token. That EOS token is how the model says "no more pulses" — and, as Part 4 shows, learning *when to stop* turns out to be one of the harder parts of the problem.

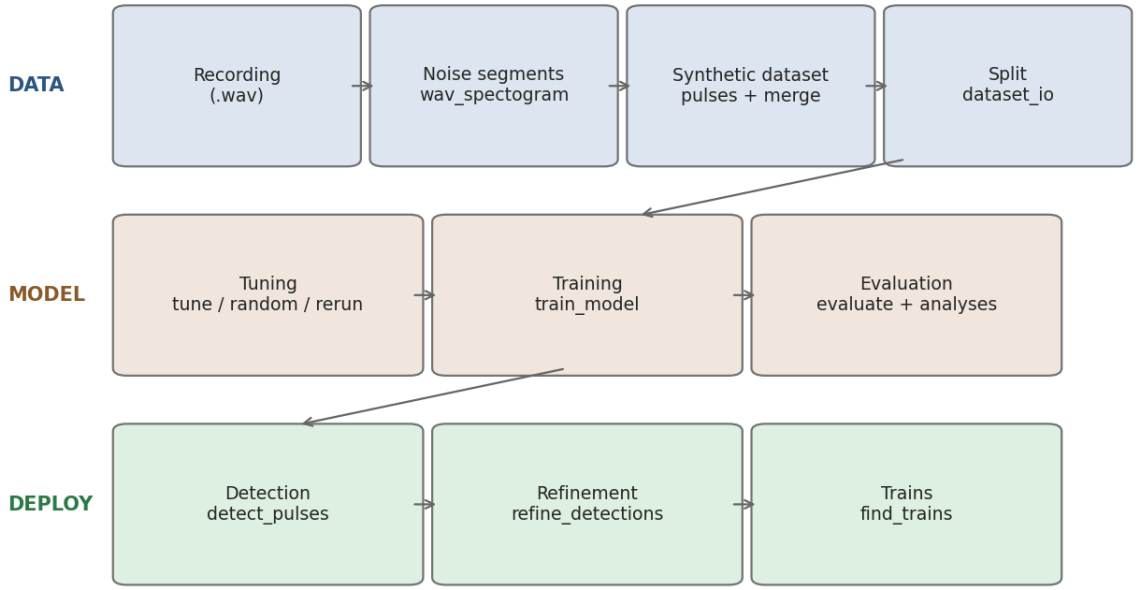

Figure 3: The full pipeline. Data preparation (blue), model development (brown), deployment (green).

Part 1 — From recording to spectrogram

1.1 The short-time Fourier transform

A Fourier transform of an entire recording tells you which frequencies are present but not *when*. The fix is to transform short slices: cut the signal into overlapping windows, transform each, and stack the results. That is the short-time Fourier transform (STFT):

$$Z(f_k, t_m) = \sum_{n=0}^{N-1} x[n + mH]\, w[n]\, e^{-i2\pi kn/N}$$

Reading the pieces: N is the window length in samples (`NPERSEG`), H is the hop between successive windows, $w[n]$ is a taper (Hann by default), and x is the signal. Each column m is one short spectrum; each row k is one frequency bin.

The taper $w[n]$ matters. Cutting a signal abruptly creates discontinuities at the window edges, and a discontinuity contains energy at all frequencies — which would smear every tone across the whole spectrum. The Hann window tapers smoothly to zero at both ends, which suppresses that smearing. It also causes each tone to occupy several bins rather than one, a fact we exploit in §1.3.

The spectrogram is then converted to decibels, because acoustic levels span many orders of magnitude and a logarithmic axis makes weak and strong pulses comparable:

$$S(f_k, t_m) = 20 \log_{10}\big(|Z(f_k, t_m)| + \varepsilon\big)$$

with a small ε preventing $\log 0$.

1.2 The resolution trade-off

The window length N controls both resolutions, in opposite directions:

$$\Delta f = \frac{f_s}{N} \qquad\qquad \Delta t = \frac{H}{f_s} = \frac{N - \text{NOVERLAP}}{f_s}$$

A longer window observes the signal for longer, so it resolves frequency more finely — but it also blurs together everything happening within that window, so time resolution worsens. You cannot improve both; this is the time–frequency uncertainty principle in its practical form.

Measured for $f_s = 50$ kHz with 50 % overlap:

`NPERSEG`	Δf	Δt
512	97.7 Hz	5.1 ms
1024	48.8 Hz	10.2 ms
2048 (used)	**24.4 Hz**	**20.5 ms**
4096	12.2 Hz	41.0 ms
8192	6.1 Hz	81.9 ms

The project uses $N = 2048$, giving 1025 frequency bins. This is a deliberate middle point: 24 Hz is fine enough to separate pulses that differ in band, and 20 ms is short compared with pulse durations of order a second. Changing it changes both axes and requires regenerating all data — these constants live in `data_config.py` precisely so that the choice is made in one place.

1.3 Sub-bin frequency estimation

This section is needed twice later (Parts 8 and 9), so it is developed here in full.

The problem. The spectrogram tells us the energy in each 24.4 Hz bin. If we simply report the loudest bin, every frequency estimate is a multiple of 24.4 Hz. How bad is that? If the true frequency falls anywhere within its bin with equal probability, the error is uniform on $\pm\Delta f/2$, and a uniform distribution on $[-a, a]$ has standard deviation $a/\sqrt{3}$, giving

$$\text{RMS error} = \frac{\Delta f}{\sqrt{12}} \approx 7.0 \text{ Hz}$$

Measured over 500 tones at random frequencies: **7.28 Hz**. So the theory is right, and 7 Hz is our starting point.

Why we can do better. Because of the window taper, a pure tone never lights up a single bin — it always illuminates its neighbours too. Crucially, *how much* it illuminates each neighbour depends on where between them the tone actually sits. Measured levels of the two neighbouring bins, relative to the peak:

True offset from bin centre	Left neighbour	Peak	Right neighbour
0.00 bins (centred)	−6.02 dB	0 dB	−6.02 dB
0.10 bins right	−7.36 dB	0 dB	−4.75 dB
0.25 bins right	−9.54 dB	0 dB	−2.92 dB
0.40 bins right	−12.04 dB	0 dB	−1.16 dB

Centred, the neighbours are symmetric. As the tone moves right, the right neighbour brightens and the left one dims, monotonically. **The asymmetry between the two neighbours encodes the sub-bin position.** We need a rule that inverts that relationship.

Choosing the rule. We have three numbers and want one. Several candidates:

- *Centroid* — treat the three levels as weights and take their weighted mean position. Simple, but it implicitly assumes a symmetric triangular peak, which a window's main lobe is not.
- *Fit a curve and take its maximum* — but which curve? It must have a peak, be determined by exactly three numbers (we have three data points), and be solvable in closed form.

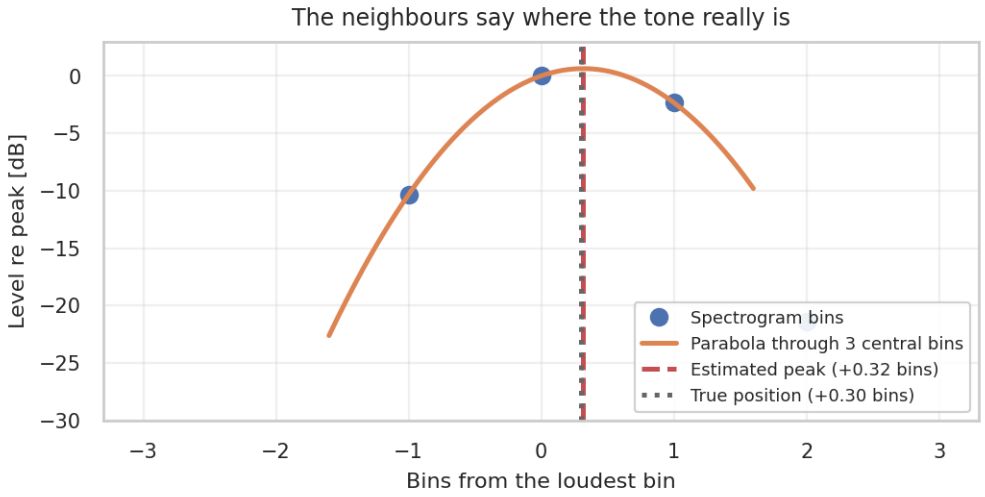

Figure 4: The three central bins (dots), the parabola through them, and its vertex. True offset +0.30 bins; estimate +0.28.

A **parabola** satisfies all three: it is the simplest curve with a peak, it has exactly three coefficients, and its vertex is a two-line formula.

There is a second, stronger reason to expect a parabola specifically, and it depends on working in decibels. A Gaussian becomes *exactly* a parabola under a logarithm:

$$20 \log_{10} \left(e^{-a(f-f_0)^2} \right) = -\frac{20a}{\ln 10} (f - f_0)^2$$

The main lobe of a Hann window is close to Gaussian near its top. So on a dB spectrogram a parabola is not an arbitrary convenient shape — it is approximately the *correct* shape. That is a testable prediction: the parabola should work better on dB values than on linear magnitudes.

Measured, over 500 tones at random sub-bin offsets:

Method	RMS error
No interpolation (bin centre)	7.28 Hz
Centroid of three bins	1.92 Hz
Parabola on **linear** magnitudes	0.93 Hz
Parabola on dB magnitudes	**0.28 Hz**

The prediction holds: dB beats linear by a factor of three, for exactly the reason above. And since the spectrogram is already stored in dB, this is the free option. Overall it is a **26-fold** improvement over reading the bin centre, for three lines of arithmetic.

The formula. Fit $y = a\delta^2 + b\delta + c$ through the three points at $\delta = -1, 0, +1$. Substituting gives $a = \frac{1}{2}(y_{-1} - 2y_0 + y_{+1})$ and $b = \frac{1}{2}(y_{+1} - y_{-1})$. The vertex of a parabola is at $\delta = -b/(2a)$, so

$$\boxed{\delta = \frac{1}{2} \cdot \frac{y_{-1} - y_{+1}}{y_{-1} - 2y_0 + y_{+1}}} \qquad \hat{f} = f_{k^*} + \delta \, \Delta f$$

Read it in two halves. The numerator $y_{-1} - y_{+1}$ is the **asymmetry**: zero when the neighbours match, positive when the left one is brighter (so the peak lies to the left). The denominator is the **curvature**: a sharply peaked lobe permits only a small correction, a broad one permits a larger one.

A limitation worth knowing. When the tone sits almost exactly halfway between two bins, the two share energy nearly equally and which one wins the `argmax` becomes arbitrary. The

estimate can then land on the wrong side. Since the true peak can never be more than half a bin from the loudest bin, $|\delta| \leq \frac{1}{2}$ always holds — so a computed $|\delta| > \frac{1}{2}$ signals an unreliable fit, and the code falls back to the bin centre in that case. In practice such columns are rare, and later stages average over dozens of them.

Part 2 — Building the dataset

2.1 Overview

Three scripts, run once per recording:

Script	Input	Output
`wav_spectogram.py`	a `.wav` recording	z-scored 5 s noise segments
`pulses.py`	noise segments	spectrograms + targets + metadata
`merge_datasets.py`	per-recording parts	one combined dataset

2.2 From recording to noise segments

Why resample. All training data must share a sampling rate, because the sampling rate fixes the spectrogram's frequency axis. `FS = 50000` Hz gives a Nyquist frequency of 25 kHz, which is the top of the usable band.

Why native rate matters. Upsampling a recording from, say, 32768 Hz to 50 kHz does not create information. The band above the original Nyquist (16.4 kHz) contains no real content — it is empty, sitting at the numerical floor of the dB conversion. A model trained on such data would learn that the upper third of the spectrum is always silent, and any pulse injected there would be trivially detectable. The script therefore **refuses** to upsample unless explicitly overridden:

```
if rate < fs:
    msg = (f"{file}: native rate {rate} Hz < target {fs} Hz. Upsampling cannot create "
           f"content above {rate / 2:.0f} Hz; the band {rate / 2:.0f}-{fs / 2:.0f} Hz would "
           f"be empty, unlike a genuine {fs} Hz recording.")
    if not allow_upsample:
        raise ValueError(msg + " Use a natively >= FS recording, or lower FS in "
                               "data_config.py and regenerate everything, or pass "
                               "allow_upsample=True to override.")
```

`detect_pulses.py` issues the same warning at deployment time, where it is a warning rather than an error — you may have no choice about the recording you are given, but you should know the top of its spectrum is not real.

Why z-score each segment. Recordings differ in gain, and gain is an artefact of the equipment, not of the ocean. Standardising each segment,

$$\tilde{x} = \frac{x - \mu}{\sigma}$$

removes it, so the model cannot key on absolute level. It also gives the pulse-injection step a fixed reference: after this, the noise has unit RMS, so an injected pulse's amplitude directly expresses its signal-to-noise ratio.

2.3 Injecting pulses

For each training example, `pulses.py` draws a random segment and injects between one and four pulses.

Amplitude and SNR. SNR is defined against the noise level before injection:

$$\mathrm{SNR}_{\mathrm{dB}} = 20 \log_{10}\left(\frac{A}{\sigma_{\mathrm{noise}}}\right) \qquad \Longrightarrow \qquad A = \sigma_{\mathrm{noise}} \cdot 10^{\mathrm{SNR}/20}$$

The factor 20 rather than 10 appears because A and σ are *amplitudes*; power goes as amplitude squared, and $10 \log_{10}(A^2/\sigma^2) = 20 \log_{10}(A/\sigma)$.

The SNR range is drawn from $U(-15, 40)$ dB. An earlier version used $U(-10, 100)$. The change matters: at 100 dB the pulse is 10^5 times the noise amplitude and detection is trivial, so more than half the training examples were teaching nothing. Concentrating the range on -15 to 40 dB puts the data where detection actually transitions from impossible to reliable, which is where a model has something to learn.

Bandwidth sampling avoids a pile-up. An earlier version drew f_1 freely and then clamped f_2 to the maximum frequency. Clamping creates a spike in the distribution: a large fraction of pulses ended at *exactly* 24990 Hz, a regularity the network can learn and exploit without learning anything about sonar. The current version samples inside the valid region instead:

```
f1 = np.random.uniform(PULSE_FREQ_MIN, PULSE_FREQ_MAX - PULSE_BW_MIN)
f2 = np.random.uniform(f1 + PULSE_BW_MIN, min(f1 + PULSE_BW_MAX, PULSE_FREQ_MAX))
```

Sweep direction. Half of all swept pulses descend, implemented by swapping the endpoints:

```
if pulse_data_array[0] != 1 and np.random.random() < 0.5:
    f1, f2 = f2, f1
```

The direction is carried implicitly by the ordering ($f_1 > f_2$ means a downsweep); no extra label is needed, and everything downstream that cares about the *band* rather than the direction sorts the pair.

2.4 The source-noise patch: a lesson in synthetic artefacts

Real sonar transmissions are accompanied by broadband noise from the transmitting platform — hull vibration, machinery. The generator models this by adding a patch of Gaussian noise under the same envelope as the chirp.

The subtle bug this originally contained. In the first version the patch was scaled to the *ambient* noise level, independently of the pulse's SNR. That has an absurd consequence at low SNR: a ping arriving 10 dB *below* the background would still be accompanied by hull noise at the background level — implying source-borne noise louder than the ping that produced it.

The practical damage was worse than the physical implausibility. At low SNR the chirp itself is invisible, but the patch still brightened the spectrogram by around 3 dB across the whole band for exactly the pulse's duration. That is a **time beacon**: a broadband step function marking $[t_{\text{start}}, t_{\text{stop}}]$ regardless of whether the pulse is detectable. A model can score excellent low-SNR recall by detecting the beacon and never learning the pulse — and the reported SNR-90 would then be a property of the generator, not the detector.

The fix follows the physics. Source noise is radiated with the ping and travels the same path, so it arrives a fixed number of decibels *below* the chirp:

$$A_{\text{patch}} = \sigma_{\text{noise}} \cdot 10^{(\text{SNR}-\Delta)/20}, \qquad \Delta \sim U(15, 30) \text{ dB}$$

Echoes get the same treatment with transmission loss applied to both components:

$$A^{\text{echo}}_{\text{chirp}} = \sigma \cdot 10^{(\text{SNR}-\text{TL})/20}, \qquad A^{\text{echo}}_{\text{patch}} = \sigma \cdot 10^{(\text{SNR}-\Delta-\text{TL})/20}$$

Now the splash is visible at high SNR, as it should be, and sinks below the noise floor at low SNR, where it can no longer act as a beacon.

The general lesson, worth carrying into any synthetic-data project: an artefact correlated with the label is indistinguishable from signal at training time, and shows up only as suspiciously good results.

2.5 Normalisation: the training/deployment contract

This is the single most important consistency requirement in the project.

At deployment, `detect_pulses.py` slides a window over a recording and z-scores each window before handing it to the model. That window *necessarily contains the pulse*, because that is what we are looking for.

The original generator z-scored the noise and *then* added the pulse. So in training the noise floor sat at a fixed absolute level, while in deployment it did not. The size of the discrepancy is computable. If a pulse of amplitude A occupies a fraction ρ of a window of unit-variance noise, the variance after injection is approximately

$$\sigma^2_{\text{after}} \approx 1 + \frac{\rho A^2}{2}$$

(the $\frac{1}{2}$ is the mean square of a sinusoid). Dividing by this standard deviation shifts the entire dB spectrogram by

$$\text{offset} = -10 \log_{10}\left(1 + \frac{\rho}{2}\, 10^{\text{SNR}/10}\right)$$

For a one-second pulse in a five-second window at SNR 30 dB, that is about -19.5 dB — a uniform shift of the whole image, far larger than the differences the model is trying to detect.

The fix is one line, applied after injection:

```
x = (x - x.mean()) / x.std()
```

Two properties make this safe. First, z-scoring is invariant to prior scaling, $z(\alpha x + \beta) = z(x)$, so the second normalisation completely absorbs the first — the result is bit-identical to a single post-injection z-score. Second, the SNR label remains valid, because a global rescale changes pulse and noise equally and SNR is a *ratio*.

A pulse example therefore passes through two z-scores. This is intentional: the first standardises the raw segment (and is the only normalisation noise-only examples ever receive), and the second establishes the deployment convention.

2.6 What the generator writes

Four files per part:

File	Contents
`{prefix}INPUT.npy`	dB spectrograms, shape $(n, 1025, T)$, memory-mapped
`{prefix}AXES.npz`	the shared time and frequency axes
`{prefix}TARGET.npy`	per-example pulse rows
`{prefix}META.npy`	per-example `[snr, distance, n_pulses, seg_idx]`

Each target row is eight numbers: a four-way one-hot type (CW, LFM, HFM, EOS) followed by the four physical quantities.

$$\underbrace{[\,c_{\text{cw}},\ c_{\text{lfm}},\ c_{\text{hfm}},\ c_{\text{eos}}\,]}_{\text{one-hot type}} \quad \underbrace{[\,t_{\text{start}},\ t_{\text{stop}},\ f_1,\ f_2\,]}_{\text{physical units}}$$

A sequence ends with a row whose type is EOS; the regression fields of that row are ignored.

Reproducibility. `generate_train(..., seed=...)` seeds the random stream, so the same call reproduces the dataset exactly. Use a **different seed per recording** — otherwise every part

receives identical pulse parameters laid over different noise, which silently reduces the effective diversity of the dataset.

2.7 Merging

`merge_datasets.py` concatenates parts, verifying that they share a spectrogram shape and axes, and streams the spectrograms through memory maps so that arbitrarily large datasets merge in constant memory.

It adds one column to `META`: **`wav_id`**, the index of the part each example came from. This is not bookkeeping for its own sake — Part 3 shows that the split cannot be done correctly without it.

```
python merge_datasets.py --prefixes hat01_,hat02n_ --out ""
```

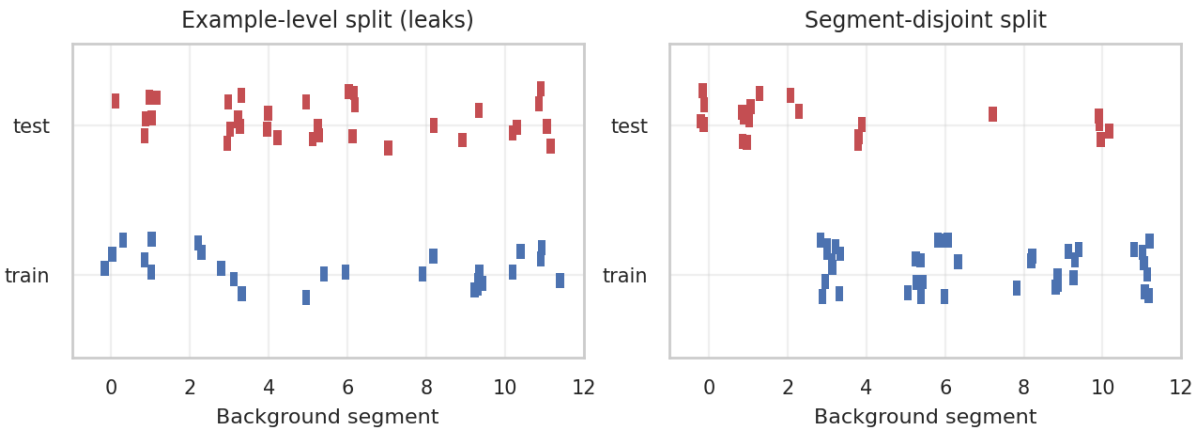

Figure 5: Left: examples from the same segment land on both sides. Right: whole segments are dealt to one side.

Part 3 — Splitting the data without cheating

3.1 Why the obvious split is wrong

The standard approach is to shuffle examples and take a third for testing. Here that silently invalidates the test set, and the reason is specific to how the data was built.

`generate_train` draws segments **with replacement**. A dataset of 17 000 examples built from 250 five-second segments reuses each background roughly **68 times**. The pulses differ each time; the ocean noise underneath does not.

Now shuffle those 17 000 examples and split off a third. Measured on exactly this setup:

Split method	Segments appearing in **both** train and test
Example-level (shuffle everything)	**250 of 250**
Segment-disjoint (group-aware)	**0 of 250**

Every single background in the test set has also been seen during training. The test set therefore measures *"can the model recognise pulses in noise it has memorised"* — which is not the question anyone wants answered.

This matters here more than in most projects because the noise is the only real part of the data. If a background contains a distinctive tonal or a passing vessel, the model can learn that texture and, with it, whatever pulses tended to accompany it.

3.2 Grouping and the group-aware split

The fix is to split at the level of the *background segment*, not the example. Every example carries a group label

$$g = \text{wav\_id} \times 10^6 + \text{seg\_idx}$$

The multiplication is not decoration: each recording numbers its segments from zero, so `seg_idx` alone would merge "segment 7 of recording A" with "segment 7 of recording B" into one group. The offset keeps them distinct.

`GroupShuffleSplit` from scikit-learn then shuffles *groups* and assigns each group entirely to one side. A segment's noise-only example and every pulse example built on it always travel together.

```
meta = np.asarray(meta)
wav_id = meta[:, 4] if meta.shape[1] > 4 else np.zeros(len(meta))
groups = wav_id.astype(np.int64) * 1_000_000 + meta[:, 3].astype(np.int64)

idx = np.arange(len(meta))
gss_test = GroupShuffleSplit(n_splits=1, test_size=cfg["test_size"], random_state=cfg["seed"])
idx_pool, idx_test = next(gss_test.split(idx, groups=groups))
gss_val = GroupShuffleSplit(n_splits=1, test_size=cfg.get("val_size", 0.15),
                            random_state=cfg["seed"])
rel_train, rel_val = next(gss_val.split(idx_pool, groups=groups[idx_pool]))
return idx_pool[rel_train], idx_pool[rel_val], idx_test
```

Two consequences to state when reporting results.

Shuffling still happens — at segment level rather than example level, and the DataLoader still shuffles examples within each split every epoch. Every recording still contributes to all three splits, through different segments. This is not a recording-level split.

Split fractions become approximate. `test_size = 0.33` now means a third of the **segments**, and segments carry different numbers of examples. Measured on the setup above, the requested 33 % produced 33.06 % of examples — close, but not exact by construction.

3.3 Three splits, not two

Split	Used for	Seen during training?
train	gradient updates	yes
validation	choosing which epoch to keep	yes, but never trained on
test	the numbers you report	no

The middle one is easy to omit and easy to misuse. If you select the best epoch using the test set, the test set has influenced your model and no longer measures generalisation — it has become a (very small) training set. The validation split exists so that selection has somewhere to happen that is not the test set.

`make_splits` peels off the test set **first**, then carves validation from the remainder, so the test set is defined independently of the validation fraction.

3.4 Memory-mapped datasets

A dataset of 75 000 spectrograms at 1025×246 float32 is roughly 300 GB. It cannot be loaded.

`MemmapPulseDataset` keeps the file on disk and reads each example on access:

```
class MemmapPulseDataset(Dataset):
    def __init__(self, input_path, targets, indices):
        self.X = np.load(input_path, mmap_mode="r")
        self.indices = np.asarray(indices)
        self.targets = torch.as_tensor(np.asarray(targets)[self.indices], dtype=torch.float32)

    def __getitem__(self, i):
        x = np.array(self.X[self.indices[i]], dtype=np.float32)
        return torch.from_numpy(x).unsqueeze(0), self.targets[i]
```

`mmap_mode="r"` maps the file into virtual memory; pages are read on demand and evicted under pressure. The targets are small enough to keep in RAM, and are exposed as `.targets` so evaluation code can read ground truth without iterating the loader. The `unsqueeze(0)` adds the channel dimension that convolutions expect: $(1025, T) \to (1, 1025, T)$.

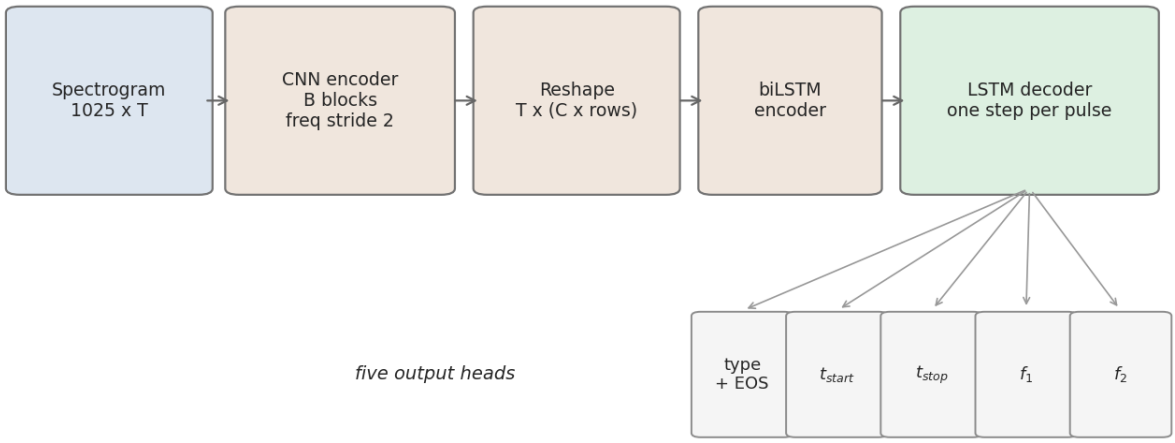

Figure 6: Encoder–decoder: the CNN and biLSTM compress the spectrogram to a summary; the decoder unrolls it into a list of pulses.

Part 4 — The model

4.1 Why this architecture

Three requirements drive it:

- **The input is an image**, with local structure (a sweep is a short diagonal streak). Convolutions exploit locality and translation invariance, so a CNN front end is natural.
- **Context spans the whole window.** Whether a streak is one pulse or two, and where it starts, depends on the surrounding seconds. A recurrent layer over the time axis provides that context.
- **The output is a variable-length list.** A recurrent decoder with a stop token handles that directly.

4.2 The CNN encoder

Each block is convolution → batch normalisation → activation → dropout. The consequential design choice is the **stride**, which is `(2, 1)`: the frequency axis is halved by each block, the time axis is left alone.

The asymmetry is deliberate. Time resolution is already good — a 20 ms spectrogram column is short compared with pulse durations — so downsampling time would throw away timing accuracy for nothing. The frequency axis has 1025 rows, which is expensive to feed to a recurrent layer, so it is reduced.

What that costs. After B blocks with frequency-stride 2, the number of surviving rows and the spectrum each row represents are

$$n_{\text{rows}} = \frac{1025}{2^B}, \qquad \text{cell width} = \frac{f_{\max}}{n_{\text{rows}}}$$

Blocks B	Rows	Cell width
0	1025	24 Hz
2	256	98 Hz
3	128	195 Hz
4	64	390 Hz

Blocks B	Rows	Cell width
5	32	780 Hz
6	16	1561 Hz

Two tones inside one cell arrive at the LSTM as the same thing. This number is a property of *your* configuration, since B is one of the parameters the hyperparameter search tunes; `snr_analysis.py` computes it from the checkpoint and marks it on the sweep-accuracy figure.

Why not simply use $B = 2$? Because the reshape that follows flattens channels and rows together, so the LSTM's input dimension is $C \times n_{\text{rows}}$:

B	Channels C	Rows	LSTM input dimension
6	512	16	8 192
5	256	32	8 192
4	256	64	16 384
3	128	128	16 384
3	64	128	8 192

Keeping frequency resolution is affordable only if the channel count is reduced at the same time. This is exactly the trade the `freq` ablation explores (§6.5), and it is why the ablation must vary both together rather than the stride alone.

4.3 Reshaping for the LSTM

```python
def forward(self, x):
    batch_size, channels, height, width = x.shape
    return x.permute(0, 3, 1, 2).reshape(batch_size, width, -1)
```

The CNN produces $(\text{batch}, C, \text{rows}, \text{time})$; an LSTM expects $(\text{batch}, \text{time}, \text{features})$. The permute moves time into position, and the reshape flattens channels and frequency rows into one feature vector per time step. After this the model treats the spectrogram as a *sequence of spectral snapshots*.

4.4 The encoder LSTM and bidirectionality

The LSTM reads that sequence and returns a final hidden state — a fixed-size summary of the whole window.

Running it **bidirectionally** means reading the sequence forwards and backwards and concatenating both states. The reason is causality: a forward-only reader, when it reaches the middle of a pulse, has not yet seen where the pulse ends. A backward pass supplies that. Since the whole window is available at once (this is offline analysis, not real-time streaming), there is no reason to withhold it.

The cost is that the decoder must accept a state of twice the width:

```python
def concatenate_bidirectional_states(h, c):
    h = torch.cat([h[0:1], h[1:2]], dim=2)
    c = torch.cat([c[0:1], c[1:2]], dim=2)
    return h, c
```

Whether this helps is an empirical question, which is why `bidirectional` is one of the ablation studies.

4.5 The decoder and its five heads

The decoder is an LSTM initialised with the encoder's summary state. At each step it emits one pulse through five separate heads:

Head	Output	Activation
classification	4 logits: CW, LFM, HFM, EOS	softmax (in the loss)
start time	1 value	sigmoid $\rightarrow [0, 1]$
stop time	1 value	sigmoid $\rightarrow [0, 1]$
start frequency	1 value	sigmoid $\rightarrow [0, 1]$
stop frequency	1 value	sigmoid $\rightarrow [0, 1]$

Why separate heads. The five quantities have different units and different natural scales. A single head would force one shared representation to serve all of them; separate heads let each specialise while sharing the decoder state that feeds them.

Why targets are scaled to $[0, 1]$**.** Raw targets span 0–5 (seconds) and 1000–25000 (Hz). Squared errors on the frequency terms would then be six orders of magnitude larger than those on the time terms, and gradient descent would optimise frequency almost exclusively. Dividing by `time_max` and `freq_max` puts all four on a common footing:

$$\tilde{t} = \frac{t}{T_{\max}}, \qquad \tilde{f} = \frac{f}{f_{\max}}$$

Why EOS is a class rather than a separate head. Stopping is mutually exclusive with emitting a pulse, so it belongs in the same softmax: the four options compete, and their probabilities sum to one. This also means "how confident is the model that a pulse is present" has a natural expression, $1 - P(\text{EOS})$, used later as the detection confidence.

4.6 Teacher forcing

During training the decoder is fed the **true** previous pulse at each step rather than its own prediction. This is teacher forcing, and it exists for two reasons: training is stable (an early mistake cannot derail the rest of the sequence), and all steps can be computed in parallel because the inputs are known in advance.

It also creates a mismatch, since at deployment the decoder must consume its own output. That mismatch is called **exposure bias**, and it is the reason model selection in this project does not use the training loss — Part 5 takes this up.

4.7 The masked loss

The loss combines classification and regression:

$$\mathcal{L} = \underbrace{\text{CE}(\hat{c}, c)}_{\text{type and EOS}} + \lambda \sum_{q \in \{t_{\text{start}},\, t_{\text{stop}},\, f_1,\, f_2\}} \text{MSE}(\hat{q}, q)$$

Why masking is necessary. Target sequences are padded to a fixed length so they can be batched. With `max_length = 10` and an average of 2.5 pulses per example, roughly **65 %** of all decoder steps are padding. Without masking, the model would spend most of its gradient budget learning to reproduce padding rows, which carry no information.

```
mask = (target_batch.sum(dim=2) != 0).float()
...
masked_loss = (per_element_loss * mask).sum() / mask.sum()
```

Dividing by `mask.sum()` rather than by the total number of elements means the loss is a mean over *real* steps, so its magnitude does not depend on how much padding a batch happens to contain.

Regression is masked further. The EOS row has no meaningful times or frequencies, so the regression terms are masked to non-EOS steps only:

```
reg_mask = mask * (1 - target_batch[:, :, 3])
```

What λ does. It sets the exchange rate between classification and regression. Too small and the model classifies well but localises poorly; too large and the reverse. It is one of the tuned hyperparameters (`lambda_reg`).

One class-balance note. With an average of 2.5 pulses per example, EOS is about **29 %** of all non-padding decoder steps — it is the single most common class. This is worth remembering when interpreting classification accuracy: a model that only ever predicted EOS would already be right 29 % of the time.

Part 5 — Measuring performance

5.1 The matching problem

The model outputs a list of pulses; the ground truth is another list. Before anything can be counted, the two lists must be paired — and neither their lengths nor their orders are guaranteed to agree.

This is why "accuracy" is not directly available. A prediction that is 0.02 s early is a success; one that is 3 s early is a false alarm plus a miss. Something has to draw that line.

The pairing rule. Sort all possible (prediction, truth) pairs by how close their start times are, and greedily accept the closest first, skipping any pulse already used. Accept a pair only if it passes two gates:

$$|t_{\text{start}}^{\text{pred}} - t_{\text{start}}^{\text{true}}| < \text{gate} \quad (0.5 \text{ s})$$

$$[f_{\text{lo}}^{\text{pred}} - \text{freq\_gate},\ f_{\text{hi}}^{\text{pred}} + \text{freq\_gate}] \cap [f_{\text{lo}}^{\text{true}},\ f_{\text{hi}}^{\text{true}}] \neq \emptyset \quad (500 \text{ Hz})$$

Why a frequency gate is needed at all. Without it, two pulses that happen to occur at the same moment in completely different bands would be matched. Measured on a concrete case — a CW prediction at 12 000 Hz, with two truths present: an LFM at 5100–6100 Hz starting 0.05 s away, and a CW at 12 000 Hz starting 0.10 s away:

Rule	Match
Time only	the LFM (closer in time, wrong band)
Time **and** frequency	**the CW** (correct)

The time-only rule pairs the prediction with a pulse 6 kHz away. The band test rejects it, and the correct pair is found.

Why the frequency gate is a padding rather than a strict overlap. A CW pulse has $f_1 = f_2$, so its band is a single line with zero width. Two CW pulses at 12 000 and 12 001 Hz would never "overlap". Padding both bands by 500 Hz gives every pulse a finite width to intersect with.

```python
def bands_overlap(a, b, tol=None):
    if tol is None:
        tol = freq_gate
    a_lo, a_hi = sorted((a["f1"], a["f2"]))
    b_lo, b_hi = sorted((b["f1"], b["f2"]))
    return (a_lo - tol) <= b_hi and (b_lo - tol) <= a_hi
```

Sorting the pair inside the function is what makes it direction-agnostic: an upsweep and a downsweep covering the same band overlap regardless of which endpoint is written first.

5.2 The metric set

From the matched pairs, `evaluate()` computes:

Detection. With TP matched pairs, FP unmatched predictions and FN unmatched truths:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Precision answers "when it reports a pulse, is there one?"; recall answers "of the pulses present, how many were found?". F_1 is their harmonic mean, which — unlike the arithmetic mean — stays low if either is low, so it cannot be gamed by sacrificing one.

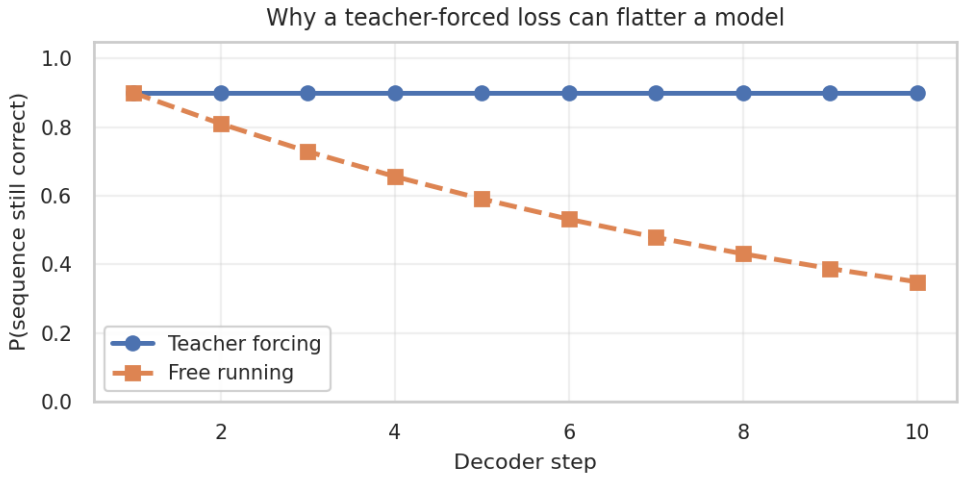

Figure 7: Teacher forcing judges each step against the truth; free running compounds its own errors.

Count accuracy. The fraction of examples where the number of predicted pulses exactly matches the truth. This measures the EOS behaviour specifically.

Count bias. The mean *signed* count error:

$$\text{bias} = \frac{1}{n}\sum_i \left(|\hat{P}_i| - |P_i|\right)$$

The sign is the point. A positive bias means systematic over-emission (EOS fires too late); negative means under-detection. Count accuracy alone cannot distinguish "sometimes too many, sometimes too few" from "consistently too many", and those call for opposite fixes.

Regression error. For each of the four quantities, over matched pairs only:

$$\text{MAE} = \frac{1}{n}\sum|\hat{q} - q|, \qquad \text{bias} = \frac{1}{n}\sum(\hat{q} - q), \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum(\hat{q} - q)^2}$$

MAE and bias must both be read. MAE measures scatter regardless of direction; bias measures systematic lean. A large MAE with near-zero bias is noise. A clear non-zero bias is a *systematic* error with a findable cause.

One such cause is worth checking in this project. The Tukey window used in generation tapers the first and last 10 % of every pulse toward silence, but the label states the sweep endpoints over the *full* nominal duration. For a swept pulse, the frequency has therefore already moved before the pulse becomes visible. A model trained on this learns to predict what it can see, which is pulled inward from the labels — producing **positive bias on f_1 and negative bias on f_2** for upsweeps. If that pattern appears in your `evaluate.py` output, part of the frequency error is mislabelling rather than model weakness, and the fix belongs in `pulses.py` (a smaller taper, or labelling the windowed endpoints), not in the architecture.

5.3 Teacher-forced loss is the wrong selection criterion

Training minimises the masked loss under teacher forcing. It is tempting to also *select* the best epoch by that loss. That is a mistake, for three separate reasons.

It cannot see error compounding. Teacher forcing resets the decoder with the truth at every step, so a model whose second pulse is garbage whenever its first pulse is slightly wrong looks perfectly healthy. Free-running generation has no such reset. A simple illustration: if each step is independently correct with probability 0.9, a teacher-forced evaluation reports 0.9 at every step, while a free-running sequence of ten steps survives with probability $0.9^{10} = 0.35$.

It cannot measure stopping. Under teacher forcing the sequence length is fixed by the target. Whether the model *stops itself* correctly — your count accuracy — simply does not exist as a quantity in that mode.

It has no notion of matching. Teacher forcing compares slot i with slot i. If a free-running model emits one spurious pulse first, every subsequent slot is misaligned and slot-wise comparison calls them all wrong, while proper matching correctly reports one false alarm and otherwise good performance.

The conclusion is standard in structured prediction: **optimise a smooth surrogate, select on the true objective.** The gradient still comes from CE + λ·MSE, because F_1 and count accuracy are not differentiable (they involve `argmax` and thresholds). But the choice of which weights to keep is made on free-running behaviour.

5.4 The AR score

The autoregressive (AR) score collapses free-running performance into one number, so epochs and configurations can be ranked. Lower is better; zero is perfect.

$$\mathrm{AR} = w_{\mathrm{det}}\,(1-F_1) + w_{\mathrm{cnt}}\,(1-\text{exact count}) + w_{\mathrm{type}}\,(1-\text{type accuracy}) + w_t\,\frac{\overline{\mathrm{MAE}}_t}{s_t} + w_f\,\frac{\overline{\mathrm{MAE}}_f}{s_f}$$

Every term is a *fraction of something wrong*, so the terms are dimensionless and comparable. Each is capped at 1 so that one catastrophic component cannot dominate the sum without limit.

Why the MAE terms need a scale. The first three terms are naturally in $[0, 1]$; the MAEs are in seconds and hertz. The scales s_t, s_f convert them by answering: *how large an error counts as one full unit of error?*

This choice is not cosmetic. The first version used the full ranges, $s_t = 5$ s and $s_f = 25\,000$ Hz. Measured contribution of the regression terms under that choice:

Time MAE	Freq MAE	AR (old scales, 5 s / 25 kHz)	AR (new scales, 0.1 s / 500 Hz)
0.02 s	50 Hz	0.006	0.060
0.05 s	200 Hz	0.018	0.180
0.10 s	500 Hz	0.040	0.400
0.20 s	1000 Hz	0.080	0.800

Under the old scales, a model with **ten times** the localisation error paid only 0.074 — less than a one-percentage-point change in type accuracy would cost. Regression was effectively invisible.

The consequence, measured. Take two candidate models. Model A localises well (0.02 s, 50 Hz) but gets 5 % of types wrong. Model B has perfect types but poor localisation (0.10 s, 500 Hz):

	Model A (good regression, 5 % type errors)	Model B (perfect types, poor regression)	Winner
Old scales	0.056	**0.040**	**B**
New scales	**0.110**	0.400	**A**

Under the original scales the score *prefers* the model with five times the frequency error. That is not a subtle bias — it is the score actively selecting the wrong model. The current scales ($s_t = 0.1$ s, $s_f = 500$ Hz) put regression on comparable footing with classification.

Weights and scales live in `CONFIG`, so they are stored in every checkpoint and a run can always be reinterpreted:

```
"ar_weights":    {"detection": 1.0, "count": 1.0, "type": 1.0, "time": 1.0, "freq": 1.0},
"ar_time_scale": 0.1,
"ar_freq_scale": 500.0,
```

Raise `count` if selection starts favouring models that over-emit. **AR values computed under different scales are not comparable**, so a change here invalidates comparison with earlier runs.

Edge cases, chosen so that degenerate situations score sensibly:

Situation	Score	Reasoning
No pulses present, none predicted	**0.0**	perfect — a correct silence
Pulses present, none matched	**5.0**	all five terms at their cap

5.5 Training

```
python train_model.py --epochs 50 --batch-size 32 --lr 1e-3
```

Each epoch: train on the train split, then evaluate twice on validation — once teacher-forced (logged) and once free-running (used for selection). The checkpoint is saved when the AR score improves.

```
ar, metrics = autoregressive_validation(
    (encoder_cnn, encoder_lstm, decoder), val_ds, cfg, device,
    batch_size=batch_size, max_examples=args.ar_max_examples)
val_ar_scores.append(ar)
row = _component_row(epoch, ar, val_loss, metrics)
...
if ar < best_ar:
    best_ar = ar
    save_best_model(epoch, ar, val_loss, row)
```

Gradient clipping. LSTMs can produce very large gradients when a long dependency suddenly matters; a single such step can destroy the weights. Clipping rescales the gradient whenever its norm exceeds a threshold:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|}\right)$$

The direction is preserved, only the length is capped.

What is written, and why each piece exists:

Output	Purpose
`best_model_epoch_E_arscore_S.pth`	weights **plus** the CONFIG they were trained with, so every later script rebuilds the exact architecture and split from the checkpoint alone
`train_losses.npy`, `val_losses.npy`	teacher-forced losses, for the correlation check
`val_ar_scores.npy`	the selection curve
`val_components.csv`	one row per epoch with every component of the AR score

The component log is the diagnostic that matters. The scalar AR score says a model got better; the components say *how*. In particular `count_bias` drifting positive across epochs means selection is starting to favour over-emitting models — visible in the log, invisible in the score.

5.6 Hyperparameter search

Why a separate tuning dataset. Searching over dozens of configurations and picking the best is itself a form of fitting. If that search runs on the training data, the selected configuration is partly fitted to it. The search therefore runs on data built from *different recordings* (`INPUT_tune.npy`).

Random search rather than grid search. With a dozen hyperparameters, a grid is combinatorially hopeless. Random sampling is also *better* than a coarse grid when only a few parameters matter: a grid wastes its budget re-testing identical values of the important parameter, while random sampling gives every trial a distinct value of each.

Two implementations exist with the same search space: `tune_hparams.py` (Ray Tune, with ASHA early stopping) and `random_search.py` (a plain loop, for when Ray will not run). Both rank by AR score and log the teacher-forced loss beside it.

ASHA early stopping. Trials report after every epoch; those in the bottom fraction are terminated. The reasoning is that a configuration far behind after five epochs rarely wins after twenty-five, so its remaining budget is better spent elsewhere.

Multi-seed reruns. A single training run confounds the configuration with the random seed. `rerun_top_configs.py` retrains the top-k configurations under several seeds and reports mean ± standard deviation, so that a difference smaller than the seed spread is not mistaken for a real effect.

```
python rerun_top_configs.py --csv tuning_results.csv --top-k 3 --seeds 0,1,2 --epochs 25
```

The data split is held fixed across seeds, so the spread measures initialisation and batch-order luck only — not split luck.

Part 6 — Evaluation and analysis

6.1 The headline numbers

```
python evaluate.py --checkpoint best_model_epoch_E_arscore_S.pth --plots
```

This runs free-running inference on the **test** split — segments no part of training or selection ever touched — and prints count accuracy, detection precision/recall/F_1, type accuracy, per-quantity regression error, and the test AR score. These are the numbers to report; everything else in this part is diagnostic.

The architecture is rebuilt from the CONFIG stored inside the checkpoint, not from the current `model.py`. This matters once you have run a hyperparameter search: `model.py` may since have been edited, and loading weights into a mismatched architecture either crashes or, worse, silently produces nonsense.

The figures written to `eval_plots/` are:

Figure	Question it answers
`count_confusion`	when the count is wrong, is it too high or too low?
`type_confusion`	which types are confused with which? (expect LFM↔HFM)
`detection`	the TP/FP/FN breakdown behind precision and recall
`error_histograms`	is the regression error scattered or systematically offset?
`pred_vs_true`	does the error depend on the value — e.g. worse at band edges?
`by_n_pulses`	does performance degrade as scenes get busier?

6.2 The bottleneck check

`by_n_pulses` deserves a note, because it tests a specific architectural hypothesis.

The encoder compresses the entire spectrogram into one fixed-size state. Every pulse the decoder emits is reconstructed from that single vector. If four pulses must be described from the same state that holds one, the state may be the limiting factor — the classic sequence-to-sequence bottleneck.

The test: stratify all metrics by the true number of pulses in the example.

- **Flat from 1 to 4** — the bottleneck is not binding; the state carries several pulses without loss.
- **Degrading with pulse count** — the state is straining, and cross-attention (letting the decoder re-read the encoder output for each pulse, rather than relying on the summary) is the standard remedy.

The zero-pulse column is separately useful: it reports false alarms per empty example, which is the training-set analogue of the deployment false-alarm rate.

6.3 SNR analysis

```
python snr_analysis.py --checkpoint best_model_...pth
```

Why per-SNR curves rather than one number. A single recall figure averages over the entire SNR range, so it depends as much on how the range was sampled as on the model. Recall as a *function* of SNR is a property of the detector.

The script bins detections by their example's SNR and reports the rate in each bin with a binomial standard error:

$$\mathrm{SE} = \sqrt{\frac{p(1-p)}{n}}$$

This is the standard deviation of a proportion estimated from n trials — essential context, because a bin containing 8 examples and a bin containing 800 deserve very different confidence.

SNR-90 is the interpolated SNR at which recall first reaches 90 %. It compresses the curve to a single operating figure, comparable across models.

The curvature analysis addresses LFM versus HFM directly. From §0.2, the two sweeps differ most at mid-pulse by

$$\Delta_{\mathrm{mid}} = \frac{(f_2 - f_1)^2}{2(f_1 + f_2)}$$

Plotting sweep-type accuracy against Δ_{mid} separates two very different explanations for confusion:

- Accuracy near chance at small Δ_{mid}, rising as it grows → **the information is not in the data**. This is expected and not a defect.
- Accuracy near chance even at large Δ_{mid} → **the model is not using available information**. This is a defect worth chasing.

The figure marks the model's frequency cell width (§4.2) as a vertical reference: to the left of that line, the two sweeps differ by less than one cell of the feature map.

The default split is `test`. Running on `all` includes the training data and produces optimistic curves; the option exists for quick checks, not for reporting.

6.4 Level robustness

```
python offset_sweep.py --checkpoint best_model_...pth --offsets=-20,-10,-5,0,5,10
```

Adds a constant dB offset to every test spectrogram and re-measures. A model that keys on *shape* is unaffected by a uniform brightness shift; one that keys on absolute level collapses.

This is a direct test of the normalisation contract from §2.5. It is also practical: real recordings differ in gain and calibration, and this quantifies the headroom.

(Note the `=` in `--offsets=-20,...`: argparse otherwise reads a leading minus as a new flag.)

6.5 Ablations

```
python ablation_study.py --studies freq,bidir,kernel --seeds 0,1,2 --epochs 25
```

Each study changes **one** design choice and holds everything else fixed — data, split, seeds, epochs, optimiser — so that any metric difference is attributable to that choice.

Study	Question
`freq`	how many CNN blocks should halve the frequency axis?
`bidir`	does reading the spectrogram backwards as well help?
`kernel`	what convolution kernel shape suits a sweep?

The `freq` study is the one connected to frequency precision (§4.2). Note that it must vary channels alongside stride, since keeping resolution while keeping width explodes the LSTM input dimension.

These metrics are computed on the *validation* split, which was also used for epoch selection within each run. That is acceptable for **ranking variants against each other**, which is what an ablation is for, but the numbers should not be quoted as generalisation performance — those come from `evaluate.py` on test.

6.6 False alarms per hour

```
python false_alarm_sweep.py --wav quiet.wav --checkpoint best_model_...pth
```

Why test-set precision is not enough. The test set is almost entirely pulse-bearing windows. Precision there answers "when the model reports a pulse, is it right?" — but an operator's question is different: *"how often will this thing cry wolf on empty ocean?"* That depends on hours of pulse-free audio, which the test set does not contain.

The script runs the full deployment chain on pulse-free recordings. Every detection is by definition a false alarm, so the rate follows directly:

$$\text{FA/hour}(\theta) = \frac{\#\{\text{detections with confidence} \geq \theta\}}{\text{hours audited}}$$

Two conditions make the number meaningful, and both are easy to violate:

- **The recording must be genuinely pulse-free**, or true detections are counted as false alarms.
- **It must not have contributed noise to training.** On memorised backgrounds the model is quieter than it will be in the field, and the measurement flatters itself.

A false-alarm rate must always be quoted with its recall. Any detector achieves zero false alarms by detecting nothing. The pair — FA/hour at threshold θ, and recall at that same θ — is the operating point, and one without the other is uninterpretable.

Part 7 — Detection on real recordings

```
python detect_pulses.py --wav recording.wav --checkpoint best_model_...pth --plot
```

7.1 Windowing

The model consumes 5-second windows; recordings are hours long. The recording is therefore cut into overlapping windows, with a hop of half a window by default.

Why overlap. A pulse straddling a window boundary appears truncated in both windows, and a truncated pulse is both harder to detect and wrongly localised. With 50 % overlap, every pulse shorter than half a window is fully contained in at least one window. The cost is that each pulse is now seen two or three times, which is what the merging step exists to undo.

Each window is z-scored before the STFT, matching the training convention exactly (§2.5).

7.2 Merging

The same pulse detected in several overlapping windows must become one detection. Two detections are merged when they satisfy **both**:

- their start times are within `gate` (0.5 s), or they overlap in time;
- their frequency bands, padded by `freq_gate` (500 Hz), intersect.

The frequency condition is what stops two simultaneous pulses in different bands from collapsing into one. The implementation also checks *every* existing cluster rather than only the most recent, so two interleaved trains in different bands each keep their own cluster:

```
for cluster in clusters:
    cluster_start = cluster[0]["t_start"]
    cluster_stop = max(d["t_stop"] for d in cluster)
    time_ok = (det["t_start"] - cluster_start <= gate or det["t_start"] < cluster_stop)
    c_lo = min(min(d["f1"], d["f2"]) for d in cluster)
    c_hi = max(max(d["f1"], d["f2"]) for d in cluster)
    band_ok = (d_lo - freq_gate) <= c_hi and (c_lo - freq_gate) <= d_hi
    if time_ok and band_ok:
        cluster.append(det)
```

Each cluster is reported once, represented by its most confident member. Clusters whose members disagree about the pulse type are flagged rather than silently resolved.

7.3 Two confidences

Each detection carries two scores from the classifier's softmax:

$$\text{confidence}_{\text{det}} = 1 - P(\text{EOS}), \qquad \text{confidence}_{\text{type}} = P(\hat{c})$$

They answer different questions — *is something there?* and *is it the type I said?* — and a detection can be high in one and low in the other. `--min-confidence` filters on the first, since screening is about presence, not type.

Neither is a calibrated probability. They come from a network trained with teacher forcing and no calibration objective, so a value of 0.9 does not mean "right 90 % of the time". They are useful for *ordering* detections and for setting a threshold empirically (via §6.6), not as probabilities.

7.4 Plotting long recordings

Two problems arise with an hours-long recording, both solved by design rather than by tuning.

A full-file spectrogram is useless and expensive. At any printable width, an hour of audio compresses each pulse to well under a pixel. Rendering it at full resolution also allocates several gigabytes. The script therefore writes:

- **one overview figure** with no spectrogram at all — each detection drawn as a vertical line spanning its band, with a confidence panel beneath — which stays readable at any duration;
- **one spectrogram crop per detection**, covering the pulse plus a few seconds of context, with the STFT computed only on the cropped samples.

A CW box hides the pulse. A CW pulse has $f_1 = f_2$, so its bounding box collapses onto the very ridge it marks. Every box is therefore expanded to a minimum height of 2 % of the visible frequency range, drawn dashed and edge-only, with the label above rather than on it:

```
lo, hi = sorted((pulse["f1"], pulse["f2"]))
min_h = max((f_hi - f_lo) * min_frac, 1.0)
if hi - lo < min_h:                        # CW, or a very narrow sweep
    pad = 0.5 * (min_h - (hi - lo))
    lo, hi = lo - pad, hi + pad
```

7.5 Reporting times

Times are reported as `h:mm:ss` with fractional seconds. The CSV keeps numeric seconds columns as well, so it stays machine-readable, and adds `t_start_hms`/`t_stop_hms` for reading.

One implementation detail is worth noting because it is a classic bug: the seconds must be rounded **before** splitting into hours, minutes and seconds. Otherwise 59.999 s formats as the invalid `0:00:60.00`.

```
seconds = round(abs(seconds), decimals)   # round BEFORE splitting
h, rem = divmod(seconds, 3600.0)
m, s = divmod(rem, 60.0)
```

Part 8 — Refinement: measuring the pulses

8.1 Why a second stage exists

The network's frequency estimate is limited by its own architecture. Section 4.2 showed that after B blocks of frequency-stride 2, one row of the feature map spans $f_{\max}/(1025/2^B)$ hertz — for $B = 6$, over 1.5 kHz. Everything inside one cell reaches the LSTM identically.

But the spectrogram that CNN consumed still has 24.4 Hz bins. **That resolution was not lost; it was discarded.** And once the detector has told us *when* a pulse occurred and roughly where, we can return to the original spectrogram and measure it there — a much easier problem than finding it in the first place.

This division is standard in passive acoustics. PAMGuard's whistle-and-moan detector, Silbido and ROCCA all pair a detector with a separate contour-measurement stage. The only unusual element here is that the detector is learned rather than energy-based.

It is worth being precise about what this stage can and cannot do:

Can	improve frequency precision, sweep direction, pulse type, start and stop times
Cannot	find a pulse the detector missed, or remove one it invented

Refinement is a *measurement* stage. Detection performance is fixed by Parts 6 and 7.

8.2 Tracking the ridge

The simplest idea first. A pulse is a bright curve in the spectrogram. Walk along the time axis and, in each column, ask which frequency bin is loudest:

$$k_m^* = \arg\max_k \; S(f_k, t_m)$$

Collecting $(t_m,\ f_{k_m^*})$ traces the pulse. This is *ridge tracking* — following the crest of a ridge in the time–frequency surface.

Refinement 1: get below the bin width. Apply the parabolic interpolation developed in §1.3, which takes the estimate from 7.28 Hz RMS to 0.28 Hz.

Refinement 2: reject columns containing no pulse. The crop deliberately includes context on both sides, and `argmax` returns *something* in every column — in the empty ones, whatever noise bin happens to be highest. Two filters remove them.

The first asks whether the peak stands out *within its own column*:

$$S(f_{k^*}, t_m) - \text{median}_k\, S(f_k, t_m) > \eta_{\text{snr}}$$

The **median** is used rather than the mean because the median is barely moved by the pulse itself — a handful of bright pulse bins cannot shift the middle value of a thousand — so it estimates the ambient level even in columns where the pulse is loud. A mean would be inflated by the very signal we are measuring against.

A worked column, from a synthetic 5000–5600 Hz sweep:

```
column at t = 0.983 s
  peak bin        5297.9 Hz at   8.8 dB
  left neighbour  5273.4 Hz at   6.9 dB
  right neighbour 5322.3 Hz at  -1.6 dB
  column median              -33.0 dB    -> peak stands 41.9 dB above
  parabola offset  delta = -0.340 bins   -> refined estimate 5289.5 Hz
```

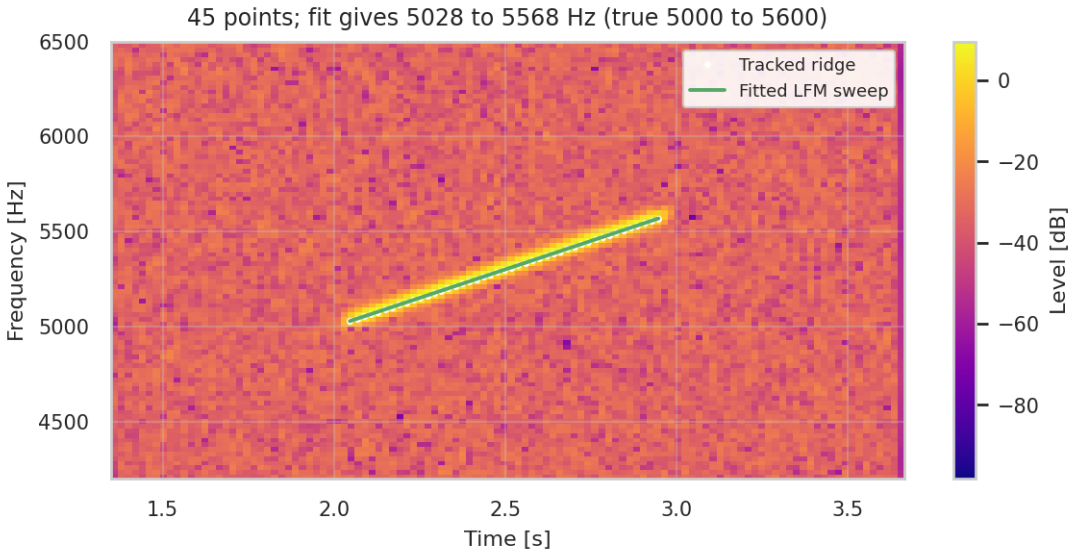

Figure 8: Ridge tracking on a detected pulse. White dots: tracked points. Green: the fitted sweep. The detection given to the refiner had a deliberately wrong band and an over-wide time span.

The left neighbour is brighter, so δ is negative and the estimate moves left, as §1.3 predicts.

The second filter compares columns with each other: drop any point more than η_{rel} dB below the loudest column in the crop. A column in the silent margin may still contain a locally prominent noise bin, but it will be far below the pulse.

Refinement 3: keep the track from wandering. If a louder signal exists elsewhere in the search band, `argmax` will jump to it. Accepted points are constrained to lie near the **median** of the surviving points, rather than chained to the previous point. Chaining is fragile — one bad point drags everything after it — whereas a robust anchor does not move.

Implementation

```
for m in range(S_band.shape[1]):
    column = S_band[:, m]
    k = int(np.argmax(column))                          # loudest bin

    if column[k] - float(np.median(S[:, m])) < snr_db:    # filter 1
        continue

    offset = 0.0                                        # sub-bin refinement
    if 0 < k < len(column) - 1:
        y0, y1, y2 = column[k - 1], column[k], column[k + 1]
        denom = y0 - 2 * y1 + y2
        if denom != 0:
            offset = 0.5 * (y0 - y2) / denom
            if abs(offset) > 0.5:                       # impossible => bad fit
                offset = 0.0

    times.append(float(t_axis[m]))
    freqs.append(float(f_band[k] + offset * df))
    levels.append(float(column[k]))

keep = level >= level.max() - rel_db                    # filter 2
```

8.3 Fitting a sweep law

The tracker produces roughly 45 points. Fitting a curve to them does two things at once: it averages away the per-point scatter, and it identifies the pulse type.

Normalise time first. With $\tau = (t - t_{\text{start}})/T$, the fitted endpoints are directly $f(0) = f_1$ and $f(1) = f_2$ — the quantities we want to report, rather than intermediate coefficients.

LFM is linear in τ, so ordinary least squares applies directly:

$$f(\tau) = a + b\,\tau$$

HFM is linear in $1/f$ (from §0.2), so fitting the *reciprocal* of the tracked frequencies makes it a linear problem too:

$$\frac{1}{f(\tau)} = \alpha + \beta\,\tau$$

This is the practical reason for preferring these two laws: both reduce to linear least squares, which has a closed-form solution and needs no iteration or initial guess.

Why the fit is so much more precise than a single column. Least squares on n points estimates two parameters. The standard error of the fitted values scales as

$$\text{SE} \sim \frac{\sigma}{\sqrt{n}}$$

With $n \approx 45$ points, the per-point scatter is divided by roughly 6.7. This is why refinement reaches sub-hertz standard errors while a single spectrogram column cannot.

Choosing between the two laws. Fit both and compare residual sums of squares:

$$\text{RSS} = \sum_m \Big(\hat{f}_m - f_{\text{model}}(\tau_m)\Big)^2$$

The smaller RSS wins. But a slope that is not significantly different from zero means a constant tone regardless of which curve fits better, so CW is tested first:

$$|b| < z\,\sigma_b \quad \Longrightarrow \quad \text{CW}$$

where σ_b is the ordinary-least-squares standard error of the slope and $z = 2$. This is a standard two-sigma significance test: is the measured slope large compared with the uncertainty on it?

Sweep direction is simply $\text{sign}(b)$ — and this is exactly where the fit beats the network. The network estimates f_1 and f_2 independently, each with its own error; for a narrow sweep, their difference is smaller than the noise on either, so the *sign* of that difference is close to a coin flip. The fit does not subtract two noisy endpoints; it estimates one slope from 45 points, and σ_b shrinks with both n and the spread in τ.

A floor on bandwidth, below which a sweep is a tone. The significance test alone is not enough to identify a CW pulse. The slope is fitted from dozens of ridge points, so its standard error can fall to a fraction of a hertz, and a drift of twenty or thirty hertz across the pulse then counts as significantly non-zero and is reported as a sweep. That drift is smaller than a single spectrogram bin: it carries no usable direction and no usable bandwidth, and labelling it LFM or HFM hands the reader two numbers that mean nothing. A pulse whose fitted frequency change is below `--min-bandwidth` (default 50 Hz, about two bins) is therefore reported as CW whatever the fit says, and the significance test is applied only afterwards.

Measured on synthetic sweeps of decreasing width:

True bandwidth	Without the floor	With the floor (50 Hz)
0 Hz (true CW)	CW	CW
20 Hz	LFM, 18.7 Hz	**CW**

True bandwidth	Without the floor	With the floor (50 Hz)
40 Hz	HFM, 36.0 Hz	**CW**
60 Hz	LFM, 55.1 Hz	LFM, 55.1 Hz
400 Hz	LFM, 360.3 Hz	LFM, 360.3 Hz

Note the 40 Hz case: without the floor it was not merely called a sweep but assigned a *curvature* type, discriminating LFM from HFM across a width smaller than two frequency bins.

The honest limit. A hyperbola is locally linear, so over a narrow fractional bandwidth the two laws fit almost identically and the RSS comparison becomes meaningless. The code measures the relative gap between them and reports `swept` instead of guessing when it falls below `--min-margin`. Direction and bandwidth remain reliable; only the curvature label is withheld.

8.4 Fitting where the pulse is, not where the detector said

An early version of this stage evaluated the fitted curve at the network's `t_start` and `t_stop`. That is wrong whenever the detected span is wider than the pulse, because the curve is then **extrapolated** beyond the tracked points.

The error grows with sweep rate. For a sweep of 10 kHz/s:

Span too wide by	Extrapolation error at each endpoint
0.1 s	1000 Hz
0.2 s	2000 Hz
0.4 s	4000 Hz

Larger than the error the whole stage exists to remove — and invisible in the numbers, since the fit itself was perfectly good.

The fix is to let the ridge define the span. The first and last tracked points become the reported start and stop times, and the curve is only ever evaluated between them:

```
ridge_start, ridge_stop = float(t[0]), float(t[-1])
fits = fit_sweeps(t, f, ridge_start, ridge_stop)
```

This makes extrapolation structurally impossible, and it yields refined **times** as a by-product. A guard rejects refinements whose ridge covers less than `--min-coverage` of the detected span, since that indicates the tracker caught only a fragment.

8.5 Cutting the ridge where the echo begins

The tracker follows energy, and an echo is the same pulse arriving again by a longer path — same band, a fraction of a second later. Nothing in the tracker distinguishes them, so the ridge runs straight on into the echo. Everything read from it is then wrong: the stop time belongs to the echo, and for a swept pulse the echo *restarts* the sweep, so the fitted slope is pulled towards a line through two sweeps instead of one, taking the endpoint frequencies with it.

This is not a rare condition. The generator injects echoes by design (§2.4), and real recordings in shallow or reverberant water contain them routinely.

Two tests are used, because each covers what the other cannot.

A gap in time. An echo arrives after the direct pulse has ended, so the ridge usually stops and resumes, leaving empty columns between them. This test does not look at frequency at all, which is why it is the one that works for a CW tone — whose echo sits at exactly the same frequency and is invisible to anything based on frequency alone.

A reversal in the sweep. A swept pulse moves its frequency one way throughout; an echo begins the sweep again, which appears as a large step back against that direction. Ordinary noise produces small reversals, so a cut is made only when the step back exceeds a quarter of the frequency span the ridge has covered.

Two failures found while building this are worth recording, because both took a working test and made it harmful:

- An earlier version compared later points against a *straight-line* fit to the early ones. A hyperbolic sweep departs from a straight line by construction, so the test cut clean HFM pulses that contained no echo. Removed: only tests that cannot be confused by the pulse's own shape are used.
- The reversal test was at first applied to every pulse. A CW tone's frequency span is its own jitter, a few tens of hertz, so a threshold set as a quarter of that is smaller than the noise and the test cut every CW pulse at the first wobble. It is now applied only when the span covers several frequency bins, which is what distinguishes a swept pulse from a tone.

Coverage (§8.4) is judged on the ridge *before* trimming, since trimming an echo is a correction rather than evidence that the tracker caught only a fragment.

Measured effect

Synthetic pulses each followed by an echo at 60 % amplitude, with the detected span deliberately wide enough to include it:

Pulse	Without trimming	With trimming
LFM up	rejected, ridge scatter 363 Hz	stop time −0.04 s, f_2 error −66 Hz
LFM down	rejected, ridge scatter 491 Hz	stop time −0.04 s, f_2 error +88 Hz
HFM up	rejected, ridge scatter 474 Hz	stop time −0.04 s, f_2 error −116 Hz
CW	stop time +1.19 **s**, frequency exact	stop time −0.04 s, frequency exact

The three swept pulses were previously discarded altogether: the echo made the ridge look too scattered to fit, so the refinement fell back to the network's own values. The CW pulse was accepted but its stop time described the echo, overstating the duration by more than the pulse itself.

Clean pulses are unaffected — LFM, HFM, a narrow 300 Hz sweep and CW without echoes all give identical results with the test on or off. Refinements where the ridge was cut are flagged in `refined_echo_trimmed`, and `--keep-echo` disables the test if you want to measure the whole arrival sequence instead.

8.6 CW pulses get one frequency

A constant tone has a single frequency, but the decoder's four regression heads are independent — nothing in the architecture ties f_1 to f_2 even when the predicted class is CW. The network learns to make them similar, but two independent estimates can differ by hundreds of hertz.

The fit resolves this properly: when the slope test returns CW, both endpoints estimate the same quantity, so they are replaced by their mean, whose standard error is smaller than either:

$$\bar{f} = \tfrac{1}{2}(f_1 + f_2), \qquad \sigma_{\bar{f}} = \tfrac{1}{2}\sqrt{\sigma_{f_1}^2 + \sigma_{f_2}^2}$$

8.7 Choosing the thresholds automatically

η_{snr} and η_{rel} resist hand-tuning, for a structural reason: a faint pulse needs a permissive threshold to produce any ridge at all, while a loud pulse beside an interfering tone needs a strict one. No single pair suits both, and there is no ground truth on a real recording to tune against.

But there is an internal quality measure. The fit residual says how well the tracked points follow a sweep law — and it needs no labels. So the code tries several threshold pairs per pulse and keeps the one with the lowest residual per point, breaking ties toward the longer ridge:

```
snr_grid = [snr_db] if snr_db is not None else [4.0, 6.0, 8.0, 12.0, 16.0]
rel_grid = [rel_db] if rel_db is not None else [12.0, 20.0, 30.0]

for s in snr_grid:
    for r in rel_grid:
        attempt = _attempt(seg, fs, det, t0, lo, hi, s, r, max_step, min_points, df)
        if attempt is None:
            continue
        if (best_attempt is None
                or (attempt["rmse"], -attempt["n_points"])
                < (best_attempt["rmse"], -best_attempt["n_points"])):
            best_attempt = attempt
```

Fifteen attempts cost about 0.1 s per pulse. Measured against fixed thresholds ($\eta_{\text{snr}} = 8$, $\eta_{\text{rel}} = 20$) on a set including two very weak pulses: fixed thresholds refined 5 of 7, the search refined **7 of 7**.

8.8 Measured accuracy

Synthetic pulses with known parameters. Each was handed a detection with ±400 Hz of frequency error, its endpoints **swapped** (so the direction was wrong), and a time span 0.15 s too wide at each end:

Pulse	True band	Refined	Error	SE	Direction	Type	Points
LFM up, narrow	5000 → 5500	5023 → 5474	+23, −26 Hz	0.10 Hz	corrected to **up**	LFM	45
LFM down, wide	9000 → 6000	8860 → 6098	−140, +98 Hz	0.50 Hz	corrected to **down**	LFM	46
CW	8000	8000	−0.4 Hz	0.02 Hz	flat	**CW**	45
HFM up	7000 → 9000	7073 → 8866	+73, −134 Hz	0.25 Hz	up	**HFM**	45
LFM, low SNR	12000 → 12800	12021 → 12774	+21, −26 Hz	0.25 Hz	up	LFM	47

Errors of tens of hertz from inputs carrying ±400 Hz; every direction recovered from a deliberately swapped input; CW and HFM correctly identified. Time errors were within 0.05 s despite spans 0.15 s too wide.

A note on interpreting the standard errors. They are the statistical uncertainty *of the fit* and are far smaller than the errors in the table. The difference is systematic: the taper (§5.2) and residual noise bias the ridge slightly. Quote the SE as fit precision, not as total accuracy.

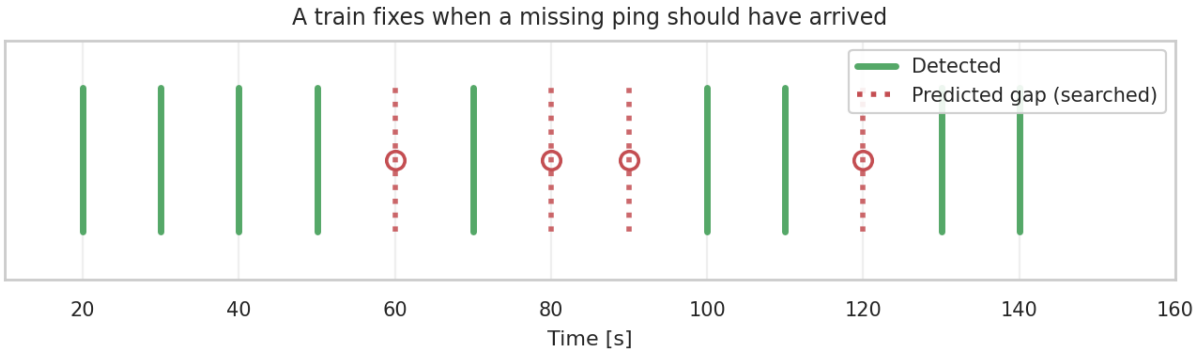

Figure 9: Detected pings (solid) establish the interval; the gaps (dotted) are where a missing ping must be if it exists.

Part 9 — Pulse trains and recovering missed pings

9.1 The idea

An active sonar transmits on a schedule, so its pings arrive at a near-constant pulse repetition interval. The per-window detector cannot use this: it sees five seconds at a time and has no way of knowing that a similar pulse arrived ten seconds earlier. Grouping detections into trains restores that context — and turns a missing ping into a *specific, testable prediction* rather than a blind search.

A train is defined by its interval and by nothing else. A transmitter may send any mixture of waveforms — CW, LFM and HFM in any order, changing from ping to ping — and those pulses still belong to one train because they arrive on one clock. This is not a hypothetical: real mid-frequency active sonar transmits packets of one to four pulses combining CW tones and FM sweeps, repeated at intervals above twenty seconds (Wiggins, 2015). Type, band and duration are therefore reported for each train but take no part in deciding membership.

Nothing assumes the pings repeat. A source with no regular interval yields no train, no prediction and no recovery, and the output equals the input. The stage is opportunistic: it improves recall on periodic sources and is inert on everything else.

9.2 What makes a set of arrivals a train

Two properties are required together, and neither suffices alone.

Tightness. The arrivals must sit close to the grid the interval implies. Measured as the standard deviation of the residuals about the *best-fitting* grid, expressed in units of one interval. Fitting the grid matters: measuring against a grid anchored on the first arrival would make the result depend on how exact the interval happens to be, and an error of a thousandth of a period — invisible at the start — becomes a large offset three hundred slots later. A long, perfectly regular train would then be judged irregular.

Occupancy. The arrivals must fill most of the slots between the first and the last. Tightness alone is not enough: a sparse grid can always be laid over a handful of scattered arrivals.

And occupancy must beat chance for that interval. Membership accepts an arrival within $\mathrm{tol} \times P$ of a slot, so each slot claims a window whose width *grows with the period*. On a busy recording a long interval therefore fills its slots whatever the arrivals do. Modelling the arrivals as randomly placed with density ρ, a slot's window is empty with probability $e^{-\lambda}$ where $\lambda = 2\,\rho\,\mathrm{tol}\,P$, so the occupancy to beat is

$$p_{\mathrm{chance}} = 1 - e^{-2\rho\,\mathrm{tol}\,P}$$

A candidate must exceed this by three standard deviations of a binomial. Without this test,

any long grid on a busy recording looks like a train — in development it produced confident "trains" at 188 s and 397 s from pure clutter.

9.3 Finding the interval

Candidates. Every gap between a pair of arrivals is a candidate interval. Pairs rather than successive gaps, because a true interval separates many pairs — neighbours, and arrivals two or three slots apart when pings in between were missed — so a train with holes still reveals its period. The gaps are binned and the bins ranked.

Ranked by density, not by count. The bins are relative, so a bin at a long period is wide in absolute terms and sweeps up unrelated gaps, while a bin at a short period is narrow. Unrelated arrivals therefore contribute counts in proportion to the period. Dividing by the period removes that bias; without the correction the search reliably preferred a *multiple* of the true interval to the interval itself.

Then each candidate is walked, not clustered. This is the crucial step. A candidate is quantised to a bin centre and may be one or two per cent from the truth. Grouping arrivals by their phase across the whole recording would need the candidate accurate to the tolerance *divided by the number of slots* — a train of eighty pings would need its interval right to about a tenth of a per cent, which no practical candidate list provides, and a candidate one per cent out scrambles the phases completely. Instead the search walks forward from a starting arrival, predicting the next slot, accepting the nearest arrival within tolerance, skipping empty slots, and **re-fitting the interval by least squares as the chain grows**. Walking only ever needs the interval right over the next step or two, and the estimate improves as the chain lengthens instead of drifting away from it.

Finally the chain is split into contiguous runs. A transmitter runs for a while and stops. Two trains of different intervals will, over a long recording, occasionally place pulses on each other's grid, and those stragglers stretch a train's apparent extent far beyond where it actually transmitted — which makes a real train look sparse and be rejected. Splitting wherever the grid stays empty for several slots keeps each genuine stretch together and leaves the stragglers as short fragments that fail the minimum length on their own.

Candidates are then taken in order of how many pulses they explain, each detection joining at most one train.

Measured performance

A single train of randomly mixed waveforms, 8 s interval, plus six unrelated detections:

	Result
Trains found	**1**, interval 8.00 s
Contents	CW x 12, LFM x 8, HFM x 7
Unrelated detections excluded	6 of 6

Three simultaneous trains among 511 detections over two hours, with 15 % of pings missed and 60 clutter detections added:

True interval	True pulses	Found	Interval recovered	Occupancy	Scatter
9 s	approx. 255	251	9.002 s	0.83	0.015
23 s	approx. 128	131	23.001 s	0.84	0.013
41 s	approx. 75	76	40.984 s	0.84	0.011

No spurious trains. Runtime 17 s.

False trains on arrivals that are not a train at all — 40 uniformly random arrival times over 600 s, 60 recordings:

Minimum pulses	Recordings given a spurious train
6	13 of 60
8 (used)	**2 of 60**

No real train was lost at either setting, which is why the bar is eight.

9.4 Predicting where a ping should be

Two kinds of prediction are made, and they rest on the same evidence.

Gaps inside the train. A slot between two confirmed pings that holds no detection.

Slots beyond both ends. A train's first and last detections are usually not where the transmission started and stopped — they are where the *detector* stopped coping. Once an interval has been established from many pings, the slot immediately past the last one is as firm a prediction as any gap inside the train, and it is checked the same way. Each ping found there becomes evidence in its own right, so the walk continues from it: a train that runs for forty more slots is followed for forty more slots. The walk ends after `--max-gap-slots` consecutive empty slots, the same rule that ends a train internally.

Two details keep the outward walk honest. The interval is **re-fitted every time a ping is recovered**, so predictions stay anchored to pings actually seen rather than extrapolating an interval estimated once and never revised — without that, the prediction drifts by its own error a little more with every slot. And a slot already holding a detection is neither searched nor counted as a miss, so a train that resumes after a detected stretch is followed straight through it.

In either case, a slot counts as a gap only if **no detection sits there** — including one that failed to join this train. Without that condition a pulse the detector had already found could be reported a second time as a recovery; in development this produced four duplicates out of seven recoveries.

Measured effect

A synthetic train of 40 pings at 5 s where the detector found only the first 15 — the case this was built for:

	Result
Pings the detector missed	25
Recovered by extension	**25 (100 %)**
Phantom pings beyond the true train	**0**
Last true ping / last recovery	215 s / 215.0 s

The walk stopped exactly at the true end of the transmission with 45 s of recording still to go. Across the five other test scenarios, whose trains do end where they appear to, extension added nothing at all — 0 extended recoveries in every one.

Recoveries are labelled `interior` or `extended` in the `recovery` column, since the evidence differs: a gap inside a train is bounded by confirmed pings on both sides, while one past the end has support on one side only. `--no-extend` turns the outward walk off.

A slot counts as a gap only if **no detection sits there** — including one that failed to join this train. Without that condition a pulse the detector had already found could be reported a second time as a recovery; in development this produced four duplicates out of seven recoveries.

9.5 Searching for the missing ping — and why matched filtering fails

The textbook answer is a matched filter: build a replica of the expected pulse and correlate. When the replica matches the signal exactly, this is the optimal detector.

It does not work here, and the reason is instructive. Coherent matched filtering is optimal only if the replica matches the true waveform *in phase* throughout, and its response falls away far faster than the parameters are known. Measured, correlating replicas against a real 5000–5500 Hz pulse:

Replica	Normalised correlation
Exact parameters	**0.970**
Off by about 25 Hz	0.198
Off by 100 Hz	0.125

A 25 Hz error — smaller than the measurement error on the endpoints, and smaller than one spectrogram bin — destroys 80 % of the response. Detection would then depend on the accuracy of the parameter estimate rather than on whether a pulse is present, which is precisely backwards.

The fix: match incoherently, in the spectrogram. Phase is what makes coherent matching brittle, and the spectrogram discards phase. So instead of correlating waveforms, the expected sweep is drawn as a **track** through the time–frequency plane, the energy along it is summed, and the result is compared with the same track displaced elsewhere in the band:

$$\text{score} = \underbrace{\frac{1}{n}\sum_{j} S\big(f_{\text{track}}(j),\, t_j\big)}_{\text{energy on the expected sweep}} \;-\; \underbrace{\text{median}_{\Delta}\left[\frac{1}{n}\sum_{j} S\big(f_{\text{track}}(j)+\Delta,\, t_j\big)\right]}_{\text{same track, displaced in frequency}}$$

The result is in dB: how much louder the predicted sweep is than the surrounding noise. Using displaced copies of the *same* track as the reference is what makes the comparison fair — the background estimate has identical shape, length and bandwidth, so any difference is due to signal rather than to the geometry of the track.

Every waveform the train uses is tried. A missing ping's type is unknown in a mixed train, so each replica is searched and the best score decides which waveform was there. When a waveform's band also moves from ping to ping, the centre frequency is scanned as well, with a step suited to the pulse: a CW tone occupies one frequency bin, so a step chosen for a wide sweep walks straight past it.

Measured separation

A synthetic train where four slots were genuinely silent and two contained pings too weak for the detector:

Slot contents	Score	Outcome at a 6 dB threshold
Genuinely silent (x 4)	about 2 dB	correctly rejected
Weak ping (x 2)	about 22 dB	**recovered**
Strong ping	about 36 dB	recovered

An order of magnitude of separation between "nothing there" and "a weak ping". Zero false recoveries; both misses found.

When two trains predict a ping at the same instant — unavoidable when interleaved waveforms share a grid — each searches with its own replica and the highest score wins, so the recovered pulse is labelled by evidence rather than by whichever train happened to be processed

first. In testing, a pulse claimed by an LFM train at +18.2 dB was correctly assigned to the HFM train at +19.0 dB.

9.6 Two limits that must be stated

Only misses inside a train are recoverable. An isolated ping, the first ping of a sequence, or a source that varies its interval has no context to extrapolate from and stays missed. This improves recall on periodic sources; it is not a general remedy for false negatives.

Recovery is circular by construction. It finds what it predicted. That is not fatal — the negative control above shows the search genuinely distinguishes signal from silence — but it does mean the threshold must be strict, recovered pings are written to a separate `source` column and never merged into the detector's own output, and **recall must be quoted both with and without this stage**.

Part 10 — Reporting, limitations and what remains

10.1 What to report, and from where

Quantity	Script	Split
Precision, recall, F_1, count accuracy, type accuracy, regression MAE	`evaluate.py`	test
Recall against SNR; SNR-90	`snr_analysis.py`	test
Sweep-type accuracy against curvature gap	`snr_analysis.py`	test
False alarms per hour, and the recall at that threshold	`false_alarm_sweep.py` + `snr_analysis.py`	held-out recording
Level robustness	`offset_sweep.py`	test
Contribution of each design choice	`ablation_study.py`	tuning validation
Refined frequency precision, direction, type	`refine_detections.py`	any
Recall with and without train recovery	`find_trains.py`	held-out recording

All three deployment scripts write CSV by default and an Excel copy with `--xlsx`.
Two rules make the difference between a defensible report and a misleading one:

- **A false-alarm rate is meaningless without its recall**, and vice versa. Quote the pair at a stated threshold.
- **Ablation and tuning numbers are for ranking**, not for reporting as performance. Those come from `evaluate.py` on test.

10.2 Diagnosing frequency error

If frequency precision is worse than desired, three checks separate causes that call for entirely different responses. Run them before changing anything, because two of the fixes force a full data regeneration and you want to make that decision once.

Check 1 — the bias signs (`evaluate.py` regression table). Positive bias on f_1 together with negative bias on f_2 (for upsweeps) is the Tukey-taper signature from §5.2: the labels state endpoints the pulse never audibly reaches. *Fix:* reduce `alpha` in `pulses.py`, or label the windowed endpoints. Regenerate and retrain.

Check 2 — error against SNR (`snr_analysis.py`). If precision at high SNR is already close to the target, the aggregate figure is dominated by the low-SNR tail and no architecture change will help. *Fix:* quote precision conditional on SNR, and use the refinement stage for the numbers you report.

Check 3 — the frequency cell (§4.2, printed by `snr_analysis.py`). If high-SNR error remains far above the target and is comparable to the cell width, the CNN is the limit. *Fix:* run `ablation_study.py --studies freq` and adopt fewer halvings, reducing channels at the same time to keep the LSTM input dimension in range.

10.3 Diagnosing false alarms

Look at the detection crops (`--plot --plot-min-confidence` at your operating threshold). Four situations, four unrelated causes:

What the crop shows	Cause	Fix
A visible transient that is not a pulse	The model has only ever seen ocean noise with no structure in it	Hard-negative mining (§10.4)
A genuine sonar-like signal from another source	Not a model error — your definition of "pulse of interest" is narrower than the training task	A scope decision: filter by band/duration/repetition, or count them as detections
Detections clustered around a real pulse	The merger split one pulse, or these are echoes	Widen `--gate` / `--freq-gate`; echoes are real arrivals
Nothing visible at all	Preprocessing mismatch between training and deployment	Check native sample rate; run `offset_sweep.py` for level sensitivity

10.4 Hard-negative mining

The most effective procedure for reducing false alarms, and it needs no labels:

1. Run detection on a pulse-free recording. Every detection is, by definition, a false alarm.
2. Cut those exact windows from the recording and z-score them.
3. Generate **two** parts from them: a noise-only part (`n_pulse_examples=0`) **and** a pulse-bearing part on the same segments.
4. Merge, retrain.

Step 3 is the one people get wrong. Adding only the noise-only part teaches the model that noise resembling that recording never contains pulses — a shortcut which suppresses false alarms *and* real detections in similar conditions. Adding pulses on the same backgrounds prevents that.

Keep the overall noise-only fraction below roughly 25 %, and after retraining check `count_bias` in `val_components.csv`: drifting negative means the correction has gone too far and the model now under-detects.

10.5 Limitations

Training data is synthetic. Every metric in this project describes performance on simulated pulses in real noise. The noise is genuine, which is the hard part; but the pulses are ideal chirps, without Doppler, multipath spreading or transmitter imperfection. **Validating on annotated real pulses is the single largest gap in the evidence.** Even a small annotated set converts simulated recall into measured recall, and answers the question no amount of synthetic tuning can: is there a domain gap?

One SNR and distance per example. All pulses in a window share them. Real scenes need not.

No overlapping pulses. The generator forbids pulses that overlap in time, so the decoder has never seen simultaneous arrivals and will not produce them — not because the output format forbids it, but because it never occurred in 17 000 training examples. Under a single-source assumption this is correct scope; with multiple sources it is a limitation.

Confidences are not calibrated. `confidence_det` orders detections usefully and supports threshold selection, but a value of 0.9 does not mean 90 % correct.

LFM/HFM at narrow bandwidth is not resolvable. This is a property of the physics (§0.2), not a defect. Both the model and the refinement stage are subject to it; the refinement stage at least *reports* it, via `--min-margin`.

Train recovery is opportunistic and circular. See §9.6.

10.6 Directions

Ordered by expected value:

Real-pulse validation, then fine-tuning. Measure first. If real pulses match the synthetic distribution, the generator is validated. If not, fine-tune on real and synthetic mixed, at low learning rate — never on real alone, which would discard the SNR coverage the synthetics provide.

Cross-attention in the decoder. If `by_n_pulses` (§6.2) shows degradation with scene complexity, letting the decoder re-read the encoder output per pulse addresses the bottleneck directly. The encoder outputs at every time step are already computed and currently discarded — only the final state is used.

A per-frequency whitened input. Normalising each frequency row by its own noise estimate before the model, as the GPL detector does, suppresses broadband transients relative to tonal energy. It targets the transient false alarms of §10.3 and is a clean ablation.

Feeding Δf to the classifier. Classification and regression are separate heads today, so nothing tells the classifier how far apart the predicted endpoints are. If swept pulses are being labelled CW *and* the refinement stage cannot fix it, this is the architectural response.

10.7 Closing note on the two-stage design

The system deliberately splits the problem in two: a learned detector that finds events in noise and handles a variable number of them, and classical estimation that measures parameters precisely once a signal is known to be present.

Each stage does what it is good at. Neural networks are strong at detection in unmodelled noise and weak at precise parameter estimation through a downsampled representation. Least-squares fitting is exact given a known signal and useless at finding one. Established passive-acoustic software is built the same way — an energy detector feeding a contour stage — and the contribution here is to show that the learned detector slots into that architecture cleanly.

Appendix A — File reference

File	Role
`data_config.py`	All data constants. Change here, and every script follows.
`wav_spectogram.py`	Recording → resampled, z-scored 5 s noise segments.
`pulses.py`	Injects synthetic pulses; writes INPUT / AXES / TARGET / META.
`merge_datasets.py`	Concatenates per-recording parts; adds the `wav_id` column.
`functions.py`	Target padding and small helpers.
`table_io.py`	Writes result tables as CSV and, on request, Excel.
`dataset_io.py`	Memory-mapped datasets and the segment-disjoint split.
`model.py`	Architecture, `CONFIG`, forward pass, masked loss.
`metrics_core.py`	Matching, the metric set, the AR score, shared inference.
`train_model.py`	Training loop; AR-based checkpoint selection; component log.
`evaluate.py`	Final report on the test split.
`tune_hparams.py` / `random_search.py`	Hyperparameter search (Ray / plain loop).
`rerun_top_configs.py`	Multi-seed reruns of the top configurations.
`ablation_study.py`	Controlled architecture variants.
`snr_analysis.py`	Recall and precision against SNR; curvature analysis.
`offset_sweep.py`	Sensitivity to a global level shift.
`false_alarm_sweep.py`	False alarms per hour against threshold.
`detect_pulses.py`	Detection on a real recording.
`refine_detections.py`	Ridge tracking and sweep fitting.
`find_trains.py`	Train grouping and recovery of missed pings.
`plot_style.py`	Shared figure conventions.
`plots.py`	Evaluation and training figures.
`plot_random_examples.py`	Dataset sanity-check grid.

Appendix B — Utility functions

Function	File	What it does
`db(z, eps)`	`detect_pulses`	$20\log_{10}(\lvert z\rvert + \varepsilon)$; the ε prevents $\log 0$.
`rms(x)`	`pulses`	Root mean square amplitude.
`load_axes(path)`	`dataset_io`	Loads the shared time and frequency axes.
`save_padded_sequences`	`functions`	Pads variable-length targets to `max_length` and stacks them.

Function	File	What it does
`strides_per_block(cfg)`	`model`	Expands a stride specification to one entry per CNN block.
`concatenate_bidirectional_states`	`model`	Joins forward and backward LSTM states for the decoder.
`hms(seconds)`	`plot_style`	Formats seconds as `h:mm:ss.ss` (rounds before splitting).
`hms_axis(ax)`	`plot_style`	Applies clock-time tick labels to an axis.
`axis_label(key)`	`plot_style`	Canonical axis label, e.g. “Start frequency [Hz]”.
`finish(ax, ...)`	`plot_style`	Applies labels, legend and rate-axis limits consistently.
`save(fig, path)`	`plot_style`	Tight-layout, save, close, report the path.
`save_table_xlsx`	`table_io`	Writes a table as Excel, numbers kept numeric so it sorts and filters.
`read_detections(path)`	`find_trains`	Reads a detections CSV, preferring refined columns.
`fmt(pulses)`	`evaluate`	Formats a pulse list for printing.

Appendix C — Command reference

```
# ---- data (once per recording) ----
#   edit and run the chain at the bottom of wav_spectogram.py
#   then, in pulses.py, with a DIFFERENT seed per recording:
#     generate_train(segments, 15000, prefix="hat01_", seed=0)
#     generate_train(segments,     0, prefix="hat02n_", seed=1)   # noise only
python merge_datasets.py --prefixes hat01_,hat02n_ --out ""
#   pad targets once (two lines at the top of train_model.py)

# ---- hyperparameter search (on a SEPARATE tuning dataset) ----
python random_search.py --input INPUT_tune.npy --target padded_sequences_tune.npy \
       --meta META_tune.npy --num-samples 40 --epochs 25 --subset 8000
python rerun_top_configs.py --csv tuning_results.csv --top-k 3 --seeds 0,1,2 --epochs 25
#   paste the printed CONFIG into model.py

# ---- train and evaluate ----
python train_model.py --epochs 50 --batch-size 32 --lr 1e-3
python evaluate.py --checkpoint best_model_epoch_E_arscore_S.pth --plots
python plots.py                                    # training diagnostics

# ---- analyses ----
python snr_analysis.py      --checkpoint best_model_...pth
python offset_sweep.py      --checkpoint best_model_...pth --offsets=-20,-10,-5,0,5,10
python false_alarm_sweep.py --wav quiet_heldout.wav --checkpoint best_model_...pth
python ablation_study.py    --studies freq,bidir,kernel --seeds 0,1,2 --epochs 25

# ---- deployment ----
python detect_pulses.py     --wav recording.wav --checkpoint best_model_...pth \
                            --min-confidence 0.95 --plot
python refine_detections.py --wav recording.wav --detections recording_detections.csv \
```

```
                          --plot --xlsx        # --keep-echo to measure the whole arrival
python find_trains.py     --wav recording.wav \
                          --detections recording_detections_refined.csv --plot --xlsx
```